



Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ Фундаментальные науки

КАФЕДРА Математическое моделирование

**РАСЧЕТНО-ПОЯСНИТЕЛЬНАЯ ЗАПИСКА
К ВЫПУСКНОЙ КВАЛИФИКАЦИОННОЙ РАБОТЕ
НА ТЕМУ:**

Сравнительный анализ
современных методов машинного обучения
в задаче машинного перевода

Студент	ФН12-816		Д.А. Нефедов
	(группа)	(подпись)	(инициалы, фамилия)
Руководитель ВКР		(подпись)	Д.А. Фетисов
			(инициалы, фамилия)
Консультант		(подпись)	(инициалы, фамилия)
Консультант		(подпись)	(инициалы, фамилия)
Нормоконтролер		(подпись)	(инициалы, фамилия)

2026 год

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	3
1 ОБЩАЯ ПОСТАНОВКА ЗАДАЧИ NMT	6
1.1 Вероятностная постановка задачи	6
1.2 Токенизация текста	7
1.3 Метрики качества перевода	8
1.4 Методы генерации перевода	10
1.5 Методы reinforcement learning для NMT	11
1.6 Методы регуляризации моделей NMT	12
2 АРХИТЕКТУРЫ МОДЕЛЕЙ NMT	15
2.1 Механизмы моделирования последовательностей	15
2.2 Encoder-Decoder архитектура	20
2.3 Архитектура только декодировщика	21
2.4 Рассматриваемые архитектуры	22
3 ЭКСПЕРИМЕНТАЛЬНОЕ СРАВНЕНИЕ NMT ПОДХОДОВ	25
3.1 Подготовка данных	25
3.2 Среда экспериментов и гиперпараметры	27
3.3 Схема подбора гиперпараметров	30
3.4 Эксперименты с токенизатором	31
3.5 Эксперименты с параметрами обучения	32
3.6 Эксперименты с архитектурами моделей	39
3.7 Эксперименты с параметрами моделей	41
3.8 Обучение лучшей конфигурации	44
ЗАКЛЮЧЕНИЕ	48
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	51
ПРИЛОЖЕНИЕ А	55

ВВЕДЕНИЕ

Нейронный машинный перевод (далее NMT) — одна из основных задач обработки естественного языка. С практической точки зрения он заключается в построении модели, которая по тексту на исходном языке формирует текст на целевом языке, сохраняя смысл, грамматическую корректность и контекст высказывания. Наряду с классическими encoder-decoder системами на основе Transformer все чаще рассматриваются decoder-only языковые модели и архитектуры с линейной обработкой последовательностей. Результаты последних соревнований WMT показывают, что крупные языковые модели уже стали важной частью систем машинного перевода, однако специализированные NMT-модели по-прежнему остаются конкурентоспособными при контролируемом обучении на параллельных корпусах [1].

Построение системы NMT включает выбор архитектуры модели, способа представления исходного и целевого текста, схемы организации перевода, набора языковых направлений, состава обучающих данных и метода обучения. Эти решения совместно определяют качество перевода, вычислительную сложность и возможность переноса модели на новые языки или домены.

С момента появления NMT было предложено несколько основных архитектурных подходов. К классическим решениям относятся рекуррентные модели LSTM с механизмом внимания [2]. Позднее основой большинства систем стали модели Transformer с механизмом самовнимания [3]. В последние годы также развиваются архитектуры, основанные на моделях пространства состояний, в частности Mamba [4]. Эти подходы по-разному работают с контекстом, поэтому их имеет смысл сравнивать не только по качеству перевода, но и по ограничениям при обработке длинных последовательностей.

Не менее важна схема организации самой модели перевода. В классическом варианте encoder-decoder encoder строит представление исходного предложения, а decoder по этому представлению формирует перевод. В decoder-only подходе исходный текст и целевой перевод рассматриваются как части одной авторегрессионной последовательности. Эти схемы могут использоваться с разными архитектурами, поэтому сравнение архитектур без учета способа организации модели дает неполную картину.

Также отличаться может и языковой охват модели. Модель может обучаться для одного направления перевода, например с одного фиксированного исходного языка на один целевой язык. Другой вариант — мультязычная модель, которая работает сразу с несколькими языками и направлениями перевода. Мультязычные модели используют общие представления между языками, но одновременно требуют более сложной балансировки данных, постановки обучения и потенциально другой логики построения перевода.

Современные работы по машинному переводу также показывают, что качество обучающих данных становится не менее важным фактором, чем выбор архитектуры [5]. Шумные параллельные пары, неверно определенный язык, неполные переводы и технические фрагменты могут ухудшать обучение даже крупных моделей. Поэтому подготовка корпуса должна рассматриваться как часть общей системы NMT, а не как вспомогательный этап. Количество техник работы с данными и их комбинаций в переводе велико: фильтрация шумных параллельных предложений, удаление дубликатов, ограничения по длине и соотношению длин предложений, различные способы аугментации данных или генерирование синтетических данных.

Различаются и подходы к обучению моделей перевода. Базовый вариант — supervised learning, при котором модель обучается на парах исходных предложений и эталонных переводов. Дополнительно могут использоваться методы дообучения с учетом автоматических метрик, человеческих оценок или reinforcement learning. Задача исследования состоит не только в описании отдельных моделей, но и в сопоставлении архитектурных, языковых, корпусных и обучающих решений в общей постановке нейронного машинного перевода.

Постановка задачи

Целью работы является сравнительное исследование подходов к построению и обучению моделей NMT. Для достижения поставленной цели необходимо решить следующие задачи:

- 1) рассмотреть общую постановку задачи NMT;
- 2) рассмотреть подходы к подготовке, фильтрации и аугментации обучающих данных;
- 3) описать основные подходы к обучению, подбору гиперпараметров и

оценке качества моделей перевода;

- 4) сравнить архитектуры LSTM, Transformer и Mamba в задачах перевода;
- 5) охарактеризовать однонаправленные и мультязычные модели перевода;
- 6) сопоставить encoder-decoder и decoder-only схемы;

1 ОБЩАЯ ПОСТАНОВКА ЗАДАЧИ NMT

1.1 Вероятностная постановка задачи

Машинный перевод заключается в построении текста на целевом языке по заданному тексту на исходном языке. В нейронном машинном переводе эта задача обычно формулируется как задача условного языкового моделирования. Перед подачей в модель текст преобразуется в последовательность токенов. В абстрактном виде токеном будем называть элемент конечного словаря: это может быть слово, часть слова, отдельный символ или специальный служебный элемент.

Пусть V_x — словарь исходного языка, а V_y — словарь целевого языка. Исходное предложение представим последовательностью токенов

$$x = (x_1, \dots, x_m), \quad x_i \in V_x,$$

а перевод — последовательностью токенов

$$y = (y_1, \dots, y_n), \quad y_t \in V_y.$$

Задача модели с параметрами θ состоит в оценке условной вероятности $p_\theta(y \mid x)$. Авторегрессионная декомпозиция этой вероятности имеет вид:

$$p_\theta(y \mid x) = \prod_{t=1}^n p_\theta(y_t \mid y_{<t}, x),$$

где $y_{<t} = (y_1, \dots, y_{t-1})$ — уже построенная часть перевода. На каждом шаге модель предсказывает следующий токен перевода, используя исходное предложение и предыдущие токены целевой последовательности.

Для обучения используется параллельный корпус $D = \{(x^{(i)}, y^{(i)})\}_{i=1}^N$, состоящий из пар исходных предложений и эталонных переводов. Параметры модели подбираются с помощью максимизации правдоподобия, что эквивалентно

минимизации отрицательного логарифма вероятности правильного перевода:

$$\mathcal{L}(\theta) = - \sum_{i=1}^N \sum_{t=1}^{n_i} \log p_{\theta} \left(y_t^{(i)} \mid y_{<t}^{(i)}, x^{(i)} \right).$$

Такая функция потерь задает штраф за низкую вероятность токенов, которые присутствуют в эталонном переводе. После обучения перевод для нового предложения можно записать как задачу поиска наиболее вероятной последовательности:

$$\hat{y} = \arg \max_y p_{\theta}(y \mid x).$$

На практике точный перебор всех вариантов невозможен, поэтому используют приближенные методы декодирования, например жадный поиск или beam search.

1.2 Токенизация текста

Нейронная модель не работает непосредственно со строкой текста. Перед обучением каждое предложение преобразуется в последовательность индексов словаря. Выбор способа токенизации влияет на длину последовательностей, размер словаря и способность модели обрабатывать редкие слова. По этой причине в машинном переводе часто используют подсловные токенизаторы, которые разбивают редкие слова на более частые фрагменты.

Одним из наиболее распространенных подходов является BPE (Byte Pair Encoding) [6]. Изначально BPE был предложен как метод сжатия данных, но в нейронном машинном переводе он используется для построения подслового словаря. Алгоритм начинает с разбиения слов на отдельные символы, после чего многократно объединяет наиболее частую пару соседних элементов. После заданного числа объединений получается словарь подсловных единиц. В результате частотные слова могут кодироваться одним токеном, а редкие слова — последовательностью более мелких фрагментов.

Другой часто используемый токенизатор называется WordPiece [7]. Он похож на BPE тем, что также строит словарь с помощью последовательного объединения более мелких единиц в более крупные. Однако критерий выбора объединения связан не только с частотой пары, но и с тем, насколько добавле-

ние нового токена улучшает вероятностную модель корпуса $\mathcal{C}$. В общем виде очередное объединение можно описать как выбор пары, дающей наибольший прирост логарифма правдоподобия:

$$(a, b)^* = \arg \max_{(a, b)} \Delta \log P(\mathcal{C}).$$

На практике WordPiece часто помечает токены, продолжающие слово, специальным префиксом. За счет этого начало слова отделяется от его внутренней части.

Токенизатор Unigram использует другой принцип, не использующий слияния подслов [8]. В нем заранее задается большой набор возможных подсловных единиц, а затем обучается вероятностная модель, в которой предложение рассматривается как результат выбора последовательности подсловных токенов. Если $u = (u_1, \dots, u_k)$ — один из возможных вариантов разбиения предложения на подслова, то его вероятность можно записать как

$$P(u) = \prod_{j=1}^k p(u_j).$$

Из всех допустимых разбиений выбирается наиболее вероятное:

$$u^* = \arg \max_{u \in S} P(u),$$

где S — множество возможных сегментаций исходной строки. В процессе обучения маловероятные подсловные единицы постепенно удаляются из словаря. Преимущество Unigram — возможность задать несколько допустимых разбиений одного и того же текста и использовать это для регуляризации обучения.

1.3 Метрики качества перевода

Качество системы машинного перевода обычно оценивают сравнением с одним или несколькими эталонными переводами. Пусть $\hat{y}$ — перевод, построенный моделью, а r — эталонный перевод. Автоматическая метрика должна численно оценить, насколько $\hat{y}$ близок к r по форме или по смыслу.

Классической метрикой машинного перевода является BLEU [9]. Она ос-

нована на точности совпадения n -грамм между переводом модели и эталоном. Для каждого порядка n вычисляется модифицированная точность p_n , в которой число совпавших n -грамм ограничивается их количеством в эталонном переводе. Итоговая оценка записывается как

$$\text{BLEU} = BP \cdot \exp \left(\sum_{n=1}^N w_n \log p_n \right),$$

где w_n — веса разных порядков n -грамм, а BP — штраф за слишком короткий перевод. Обычно используют $N = 4$ и равные веса. Метрика BLEU удобна для сравнения систем на корпусном уровне, но плохо учитывает синонимы и другие допустимые переформулировки.

Метрика chrF, или CHRF, оценивает совпадение символьных n -грамм [10]. В отличие от BLEU, она учитывает не только точность, но и полноту. Пусть P_{chr} — средняя точность по символьным n -граммам, а R_{chr} — средняя полнота. Тогда оценка имеет вид:

$$\text{chrF}_\beta = \frac{(1 + \beta^2)P_{chr}R_{chr}}{\beta^2P_{chr} + R_{chr}}.$$

Так как сравнение выполняется на уровне символов, chrF лучше реагирует на частичные совпадения словоформ. Это особенно важно для языков с развитой морфологией, где разные формы одного слова могут иметь общие символьные фрагменты.

Более современным подходом является COMET [11]. В отличие от BLEU и chrF, эта метрика является обучаемой нейронной моделью. Она получает на вход исходное предложение, перевод модели и эталонный перевод, после чего предсказывает оценку качества:

$$q = f_\phi(x, \hat{y}, r),$$

где f_ϕ — обученная модель оценки, а q — предсказанная численная оценка. COMET обучается на данных с человеческими оценками переводов, поэтому может учитывать смысловую близость даже в тех случаях, когда поверхностное совпадение слов невелико. При этом такая метрика сложнее в использовании, так как требует отдельной предобученной модели и зависит от данных, на

которых эта модель была обучена.

1.4 Методы генерации перевода

После обучения авторегрессионная модель задает распределение вероятностей следующего токена $p_\theta(y_t \mid y_{<t}, x)$. Для получения конкретного перевода необходимо выбрать правило декодирования. Детерминированные методы, такие как жадный поиск или beam search, выбирают наиболее вероятные продолжения и обычно применяются при итоговой оценке качества. Для техник reinforcement learning (подробнее в следующем разделе) требуется получать несколько различных вариантов ответа, поэтому чаще используется стохастическая генерация. Несмотря на то, что стохастическая генерация текста более применима к LLM моделям, которым нужно быть “творческими”, такой метод применим и к NMT задачам, где “разброс” оптимального ответа куда ниже.

Одним из параметров стохастической генерации является temperature. Пусть z_i — logit токена i перед softmax. При температуре T распределение следующего токена имеет вид:

$$p_i(T) = \frac{\exp(z_i/T)}{\sum_j \exp(z_j/T)}.$$

При $T < 1$ распределение становится более резким, и модель чаще выбирает наиболее вероятные токены. При $T > 1$ распределение сглаживается, поэтому генерация становится более разнообразной, но возрастает риск ошибок.

Другой распространенный прием — nucleus sampling, или top-p sampling [12]. На каждом шаге выбирается минимальное множество наиболее вероятных токенов S_p , сумма вероятностей которых не меньше заданного порога p :

$$S_p = \min \left\{ S : \sum_{i \in S} p_i \geq p \right\}.$$

После этого вероятности токенов вне S_p зануляются, а оставшееся распределение нормируется заново. В отличие от top-k sampling, где число допустимых токенов фиксировано, top-p динамически меняет размер множества кандидатов в зависимости от уверенности модели.

Как правило, *temperature* и *top-p sampling* используют вместе, чтобы ограничить маловероятные токены и при этом сохранить разнообразие генерации. Фрагмент реализации такого выбора следующего токена приведен в приложении А.

1.5 Методы reinforcement learning для NMT

После обучения по параллельному корпусу модель перевода можно дополнительно оптимизировать не только по *token-level cross entropy*, но и по внешней оценке качества построенного перевода. В такой постановке генерация перевода рассматривается как последовательное принятие решений: на каждом шаге модель выбирает следующий токен, а после получения полной последовательности получает *reward* за качество результата.

Пусть $\hat{y}$ — перевод, сгенерированный моделью для исходного предложения x , а r — эталонный перевод. Reward-функцию можно задать через автоматические метрики качества:

$$R(x, \hat{y}, r) = \alpha_1 \text{BLEU}(\hat{y}, r) + \alpha_2 \text{chrF}(\hat{y}, r) + \alpha_3 \text{COMET}(x, \hat{y}, r),$$

где α_1 , α_2 и α_3 задают вклад отдельных метрик. В простейшем случае в качестве *reward* можно использовать одну метрику, например COMET, либо взвешенную комбинацию BLEU, chrF и COMET.

Одним из методов такой оптимизации является GRPO (Group Relative Policy Optimization) [13]. Для одного входного предложения модель генерирует группу вариантов перевода, обычно с помощью стохастического декодирования с *temperature* и *top-p*, после чего каждый вариант оценивается reward-функцией. Разные варианты внутри группы дают локальное сравнение качества переводов для одного и того же исходного предложения. Вместо обучения отдельной value-модели GRPO сравнивает *reward* отдельного ответа со средним *reward* внутри группы. Если вариант лучше среднего, его вероятность увеличивается; если хуже среднего, уменьшается. Такая интерпретация делает метод удобным для задач, где качество ответа можно оценить внешней метрикой, но трудно задать правильное действие на каждом отдельном шаге генерации.

В упрощенном виде оптимизируемую функцию можно записать как

clipped policy-gradient loss с KL-регуляризацией относительно опорной модели:

$$\mathcal{L}_{GRPO}(\theta) = -\mathbb{E}_{x, \hat{y}} [\min(\rho_{\theta} A, \bar{\rho}_{\theta} A) - \beta D_{KL}(\pi_{\theta} \parallel \pi_{ref})].$$

Здесь $\rho_{\theta} = \pi_{\theta}(\hat{y} \mid x) / \pi_{old}(\hat{y} \mid x)$ — отношение вероятностей ответа в новой и старой политиках, $\bar{\rho}_{\theta} = \text{clip}(\rho_{\theta}, 1 - \varepsilon, 1 + \varepsilon)$ — ограниченное отношение вероятностей, A — групповая advantage-оценка, вычисленная из reward вариантов внутри одной группы, ε — параметр ограничения шага обновления, β — вес KL-штрафа, а π_{ref} — опорная модель. Знак минус связан с тем, что при обучении обычно минимизируется loss, хотя сама policy-gradient часть соответствует максимизации ожидаемого reward. Фрагмент реализации GRPO loss приведен в приложении А.

BLEU можно использовать как часть reward, но он плохо подходит в качестве единственного сигнала оптимизации. Метрика основана на поверхностном совпадении n -грамм, слабо учитывает допустимые переформулировки и на уровне отдельных предложений дает шумный и разреженный сигнал. В исследованиях reinforcement learning для NMT также отмечалось, что оптимизация непосредственно под BLEU может давать нестабильное обучение и не всегда приводит к улучшению человеческой оценки перевода [14]. Поэтому для reward в переводе предпочтительно использовать более устойчивые сигналы, например chrF и COMET, либо их комбинацию с BLEU.

1.6 Методы регуляризации моделей NMT

При обучении моделей NMT число параметров обычно велико, а параллельный корпус ограничен по размеру и качеству. Поэтому модель может не только изучать общие закономерности перевода, но и запоминать частные особенности обучающих примеров. Для уменьшения такого эффекта используют методы регуляризации: они изменяют процесс обучения так, чтобы модель давала более устойчивые предсказания на новых данных.

Одним из распространенных методов является dropout [15]. Пусть h — вектор активаций некоторого слоя, $h = (h_1, \dots, h_d)$, а p — вероятность за нуления элемента. Во время обучения случайно выбирается маска

$$m_i \sim \text{Bernoulli}(1 - p), \quad i = 1, \dots, d.$$

После этого вместо исходного вектора h в следующий слой передается вектор

$$\tilde{h} = \frac{m \odot h}{1 - p},$$

где $\odot$ обозначает покомпонентное умножение. Такое масштабирование сохраняет математическое ожидание активаций: $\mathbb{E}\tilde{h}_i = h_i$. На каждой итерации обучения сеть фактически работает с разными подмоделями, из-за чего отдельные нейроны меньше зависят от фиксированного набора соседних признаков. В NMT dropout применяют внутри слоев модели: к embeddings, выходам attention и feed-forward блоков, а также к промежуточным представлениям между слоями.

Другой подход связан с ограничением величины весов модели. Если параметры модели обозначить через w , то при обычной максимизации правдоподобия выбираются веса, при которых велика вероятность обучающих данных $p(D | w)$. В байесовской постановке рассматривается другая величина — апостериорная плотность весов при условии данных:

$$p(w | D) = \frac{p(D | w)p(w)}{p(D)}.$$

Так как $p(D)$ не зависит от w , получается оценка максимума апостериорной вероятности (maximum a posteriori, MAP)

$$\hat{w}_{MAP} = \arg \max_w (\log p(D | w) + \log p(w)).$$

В MAP-подходе максимизируется апостериорная плотность весов $p(w | D)$, а не только правдоподобие данных $p(D | w)$. Если в качестве априорного распределения взять нормальное распределение $p(w) = \mathcal{N}(0, \sigma^2 I)$, то

$$\log p(w) = -\frac{1}{2\sigma^2} \|w\|_2^2 + C.$$

Отсюда получается функция потерь с ℓ_2 -регуляризацией [16]:

$$\mathcal{L}_{reg}(w) = \mathcal{L}(w) + \frac{\lambda}{2} \|w\|_2^2.$$

В этом случае большие веса штрафуются квадратичной добавкой и заранее счи-

таются менее предпочтительными. В градиентном спуске это приводит к обновлению

$$w_{t+1} = (1 - \eta\lambda)w_t - \eta\nabla_w \mathcal{L}(w_t),$$

где η — шаг обучения. Первый множитель постепенно уменьшает веса, поэтому в оптимизаторах этот прием называют *weight decay*. В современных адаптивных оптимизаторах *weight decay* часто реализуют как отдельный шаг уменьшения весов, а не как часть градиента функции потерь [17].

Еще одним способом регуляризации является *label smoothing* [18]. В стандартном обучении правильный token кодируется *one-hot* распределением: вся вероятность относится к одному элементу словаря. Для NMT такая постановка бывает слишком жесткой, поскольку один и тот же фрагмент текста может иметь несколько допустимых переводов. Пусть $K = |V_y|$ — размер целевого словаря, а y_t — правильный token на шаге t . При *label smoothing* целевое распределение заменяется сглаженным распределением:

$$\tilde{q}_t(k) = (1 - \varepsilon)\mathbf{1}\{k = y_t\} + \frac{\varepsilon}{K},$$

где ε — коэффициент сглаживания. Функция потерь тогда записывается как

$$\mathcal{L}_{LS} = - \sum_t \sum_{k=1}^K \tilde{q}_t(k) \log p_\theta(k \mid y_{<t}, x).$$

Label smoothing ограничивает чрезмерную уверенность модели в единственном токене и обычно делает распределение выходных вероятностей более устойчивым.

2 АРХИТЕКТУРЫ МОДЕЛЕЙ NMT

В нейронном машинном переводе архитектура модели определяет два связанных аспекта: способ представления контекста внутри последовательности и способ передачи информации от исходного текста к порождаемому переводу. Первый аспект задается вычислительными блоками, из которых строятся слои модели; второй — общей схемой условной генерации. Поэтому сравнение архитектур целесообразно начинать с механизмов обработки последовательностей, а затем переходить к encoder-decoder и decoder-only постановкам.

2.1 Механизмы моделирования последовательностей

При обработке текста модель должна учитывать зависимости между токенами, расположенными на разных позициях последовательности. От того, как именно передается такой контекст, зависят выразительность модели, возможность параллельного обучения и вычислительная сложность.

В рассматриваемых архитектурах используются три механизма моделирования последовательностей: рекуррентное состояние LSTM, взвешенный доступ к представлениям через attention и избирательное обновление состояния в Mamba.

LSTM. LSTM является рекуррентной архитектурой, предназначенной для обработки последовательностей и уменьшения проблемы затухающих градиентов [19]. На каждом шаге слой получает входной вектор z_t , предыдущее скрытое состояние h_{t-1} и состояние памяти c_{t-1} . Обновление памяти управляется

входным, забывающим и выходным вентилями:

$$\begin{aligned}
i_t &= \sigma(W_i z_t + U_i h_{t-1} + b_i), \\
f_t &= \sigma(W_f z_t + U_f h_{t-1} + b_f), \\
o_t &= \sigma(W_o z_t + U_o h_{t-1} + b_o), \\
\tilde{c}_t &= \tanh(W_c z_t + U_c h_{t-1} + b_c), \\
c_t &= f_t \odot c_{t-1} + i_t \odot \tilde{c}_t, \\
h_t &= o_t \odot \tanh(c_t).
\end{aligned}$$

Смысл gating-механизма состоит в управляемом обновлении памяти: вентиль забывания f_t определяет, какая часть прежнего состояния c_{t-1} сохраняется, входной вентиль i_t регулирует запись новой кандидатной информации $\tilde{c}_t$, а выходной вентиль o_t задает, какая часть памяти становится видимой в скрытом состоянии h_t . Благодаря такому разделенному управлению LSTM может удерживать существенную информацию на нескольких шагах последовательности и подавлять нерелевантные обновления. Главное ограничение рекуррентного блока состоит в последовательной обработке токенов: шаг t зависит от состояния шага $t - 1$. Структура вычислений LSTM-ячейки показана на рисунке 1.

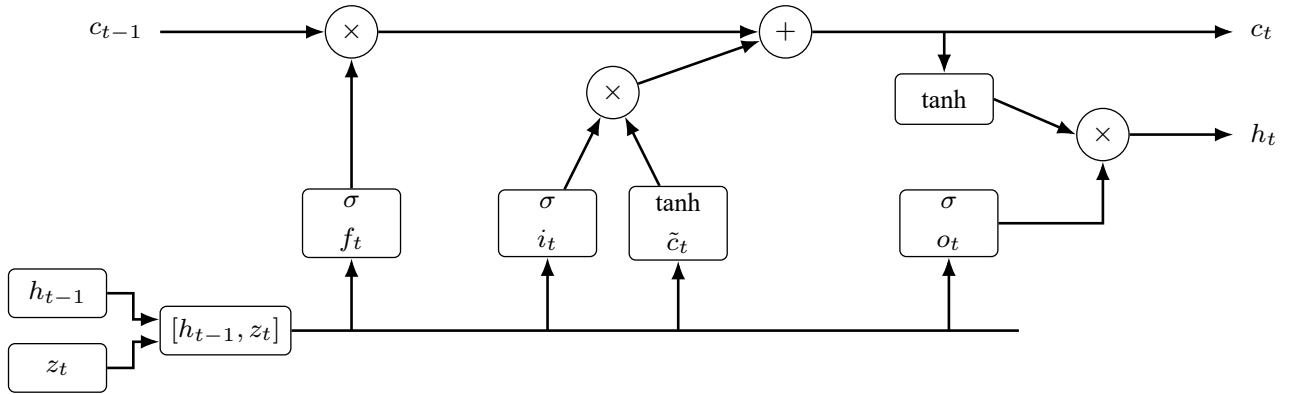


Рисунок 1 – Блок-схема LSTM-ячейки

Attention. В классической seq2seq-схеме перевода encoder на основе LSTM последовательно обрабатывает исходное предложение и передает decoder последнее скрытое состояние как контекстный вектор [20]. Такая схема связывает качество перевода с тем, насколько один вектор способен сохранить информацию обо всей входной последовательности, что особенно ограничивает модель на длинных предложениях.

Для уменьшения зависимости decoder от единственного контекстного вектора был предложен механизм внимания, или attention [2, 3]. Он дает модели прямой доступ к набору представлений encoder, поэтому информация об исходном предложении не должна полностью сжиматься в последнее скрытое состояние. В общем виде attention можно описать через три набора векторов: запросы Q , ключи K и значения V . Запрос определяет, какая информация требуется текущей позиции, ключи используются для вычисления близости с другими позициями, а значения усредняются с весами, полученными после нормализации этих близостей. В варианте, используемом в Transformer, веса вычисляются по скалярному произведению запросов и ключей:

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V,$$

где d_k — размерность ключей, а результатом является взвешенная сумма значений V . В self-attention запросы, ключи и значения строятся из одной и той же последовательности, поэтому токены внутри предложения обмениваются информацией друг с другом. В cross-attention запросы поступают из decoder, а ключи и значения — из encoder; так decoder получает доступ к представлениям исходного предложения. Схема вычисления attention приведена на рисунке 2.

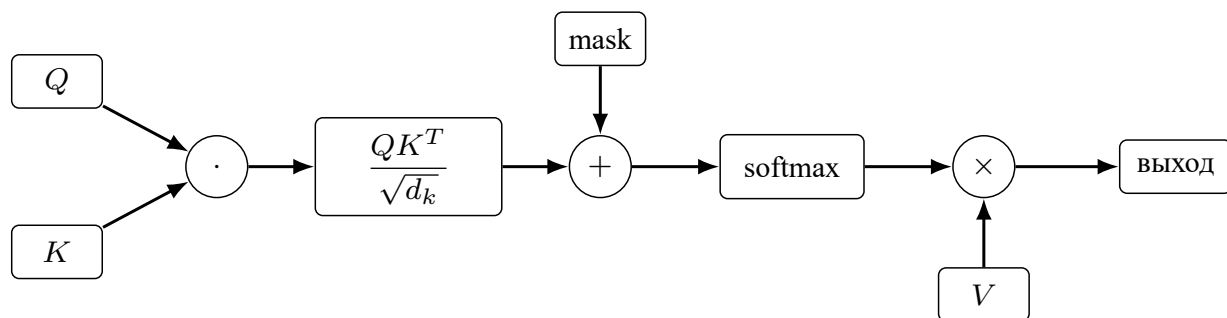


Рисунок 2 — Схема вычисления attention

Multi-head attention. Одна attention-операция формирует единое взвешенное представление контекста, из-за чего разные типы зависимостей между токенами могут смешиваться. Например, в одном представлении могут одновременно требоваться локальные связи между соседними словами, дальние синтаксические зависимости и соответствия между словами исходного предложения и перевода. Для параллельного учета таких зависимостей было предложено “многоголовое” внимание: в этом механизме вычисления разбиваются на несколько

параллельных attention-голов:

$$\text{MHA}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_H)W^O,$$

Каждая голова имеет собственные проекции Q , K и V . За счет этого модель одновременно учитывает разные типы связей между токенами. Структура multi-head attention представлена на рисунке 3.

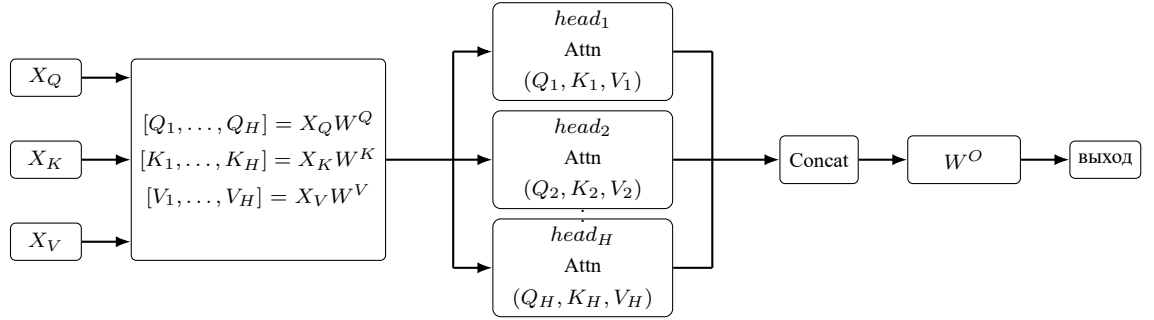


Рисунок 3 – Блок-схема multi-head attention

В self-attention выполняется $X_Q = X_K = X_V$. В cross-attention последовательность запросов X_Q поступает из decoder, а X_K и X_V строятся по выходам encoder. Фрагмент реализации multi-head attention с FlashAttention приведен в приложении А.

Позиционные представления. Self-attention не содержит встроенного порядка токенов. Если к входным векторам не добавить информацию о позиции, слой attention обрабатывает последовательность как набор элементов: перестановка токенов приводит к той же перестановке выходных представлений. Поэтому Transformer использует позиционные представления, которые передают номер токена в последовательности.

В оригинальной архитектуре Transformer применяется absolute positional encoding [3]. Для позиции pos и координаты i задаются синусоидальные признаки:

$$PE_{pos,2i} = \sin\left(\frac{pos}{10000^{2i/d_{model}}}\right),$$

$$PE_{pos,2i+1} = \cos\left(\frac{pos}{10000^{2i/d_{model}}}\right),$$

где d_{model} — размерность векторного представления. Полученный позиционный вектор имеет ту же размерность, что и embedding токена, и складывается с

ним перед подачей в первый слой модели:

$$x_{pos} = e_{pos} + PE_{pos}.$$

Такая схема передает в модель абсолютный номер позиции, а синусоидальная форма задает разные частоты по координатам вектора.

Вместо абсолютной позиции в модель можно передавать информацию об относительной позиции. Так, например, работает rotary positional embeddings (RoPE) [21]. В RoPE позиционный вектор не прибавляется к embedding. Вместо этого позиция задает поворот пар координат векторов query и key перед вычислением attention. Для пары координат (x_{2i}, x_{2i+1}) преобразование можно записать как

$$\begin{pmatrix} x'_{2i} \\ x'_{2i+1} \end{pmatrix} = \begin{pmatrix} \cos(pos \theta_i) & -\sin(pos \theta_i) \\ \sin(pos \theta_i) & \cos(pos \theta_i) \end{pmatrix} \begin{pmatrix} x_{2i} \\ x_{2i+1} \end{pmatrix},$$

где θ_i задает частоту для соответствующей пары координат. Так как поворот применяется к Q и K , позиционная информация попадает непосредственно в скалярные произведения, по которым вычисляются attention-веса. Скалярное произведение повернутых векторов зависит не только от их содержания, но и от расстояния между позициями. Поэтому RoPE вводит относительную позиционную информацию без отдельного добавления позиционного embedding к входу. Фрагменты реализации absolute positional encoding и RoPE приведены в приложении А.

Mamba. Self-attention дает прямой доступ к другим позициям последовательности, но требует вычисления попарных связей между токенами. Из-за этого время и память растут квадратично с длиной последовательности, что становится ограничением при работе с длинными контекстами. Mamba была предложена как альтернатива attention-блокам: она относится к моделям пространства состояний с избирательным обновлением состояния [4]. В упрощенном виде такой блок можно записать как

$$s_t = A_t s_{t-1} + B_t z_t, \quad r_t = C_t s_t,$$

где s_t — скрытое состояние, z_t — входной вектор, а r_t — выход блока. Ключевая особенность Mamba — зависимость параметров обновления от текущего входа, за счет которой модель избирательно сохраняет или забывает информацию. В отличие от self-attention, такой блок обрабатывает последовательность за линейное время по ее длине, что делает его привлекательным для длинных контекстов. Основные вычислительные ветви Mamba-блока показаны на рисунке 4.

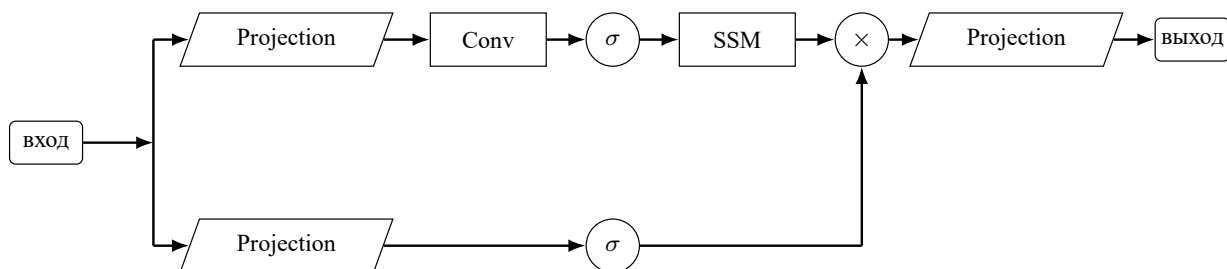


Рисунок 4 — Блок-схема Mamba-блока

Блоки LSTM, attention и Mamba определяют вычислительное ядро моделей NMT, но сами по себе не задают полную схему перевода. Архитектура системы определяется тем, как эти блоки объединяются в процесс условной генерации. В NMT для этого используются две основные постановки: encoder-decoder схема и decoder-only формулировка.

2.2 Encoder-Decoder архитектура

Архитектура encoder-decoder описывает перевод как две связанные стадии: кодирование исходного предложения и авторегрессионное порождение целевого предложения. В принятой постановке исходная последовательность имеет вид $x = (x_1, \dots, x_m)$, а целевая последовательность — $y = (y_1, \dots, y_n)$. Encoder преобразует исходные токены в набор скрытых представлений:

$$H = (h_1, \dots, h_m) = \text{Enc}_\phi(x_1, \dots, x_m),$$

где H можно рассматривать как память модели об исходном предложении. Decoder строит распределение следующего целевого токена, используя уже по-

строенный префикс перевода и представления encoder:

$$\pi_t = \text{Dec}_\psi(y_{<t}, H), \quad p_\theta(y_t \mid y_{<t}, x) = \pi_t(y_t),$$

где $\theta = (\phi, \psi)$ — параметры всей модели. Тогда условная вероятность перевода записывается как

$$p_\theta(y \mid x) = \prod_{t=1}^n p_\theta(y_t \mid y_{<t}, x).$$

В такой формулировке encoder отвечает за представление исходного текста, а decoder — за построение перевода. Связь между ними может быть реализована по-разному: через последнее состояние encoder, через attention над всеми состояниями encoder или через более сложные механизмы доступа к памяти. Для NMT особенно важен вариант с attention, поскольку decoder получает возможность на каждом шаге выбирать релевантные части исходного предложения.

2.3 Архитектура только декодировщика

В decoder-only постановке отдельный encoder отсутствует. Исходное предложение и целевой перевод записываются в одну авторегрессионную последовательность. Например, можно задать префикс

$$q(x) = (\langle src \rangle, x_1, \dots, x_m, \langle tgt \rangle),$$

после которого модель должна продолжить последовательность токенами перевода. Условная вероятность в этом случае имеет вид

$$p_\theta(y \mid x) = \prod_{t=1}^n p_\theta(y_t \mid q(x), y_{<t}).$$

В этой схеме используется каузальная маска: каждый токен может обращаться только к предыдущим токенам общей последовательности. Во время обучения функция потерь обычно считается только по целевым токенам, а исходные токены играют роль условия или промпта. Преимущество decoder-only подхода — единая формулировка задачи генерации, а недостаток — отсутствие

отдельного представления исходного предложения и явного cross-attention между исходной и целевой последовательностями.

Сочетание encoder-decoder и decoder-only постановок с разными механизмами моделирования последовательностей позволяет сравнивать архитектуры по двум признакам: где хранится представление исходного предложения и каким блоком моделируются зависимости между токенами.

2.4 Рассматриваемые архитектуры

LSTM Encoder-Decoder. LSTM Encoder-Decoder используется как базовая рекуррентная архитектура NMT [20, 2]. Encoder читает исходное предложение слева направо и после каждого токена обновляет скрытое состояние. Decoder строит перевод также последовательно: очередной токен выбирается по предыдущему целевому токenu, состоянию decoder и контексту исходного предложения.

Без attention такая схема вынуждена передавать всю информацию decoder через последнее состояние encoder. Для длинных предложений это становится узким местом: детали начала предложения могут быть вытеснены последующими токенами. Attention над состояниями encoder ослабляет это ограничение, поскольку decoder получает доступ не только к последнему состоянию, но и к представлениям всех исходных позиций. При этом рекуррентная природа LSTM остается: токены нельзя обработать полностью параллельно, поэтому обучение и декодирование хуже масштабируются на современных ускорителях.

Transformer Encoder-Decoder. Transformer Encoder-Decoder сохраняет явное разделение encoder и decoder, но заменяет рекуррентное чтение последовательности self-attention слоями [3]. Encoder строит контекстные представления исходных токенов, где каждая позиция может учитывать остальные позиции предложения. Decoder использует causal self-attention для уже построенной части перевода и cross-attention для обращения к выходам encoder.

Такое разделение удобно для машинного перевода. Encoder отвечает за исходное предложение и не ограничен каузальной маской, а decoder порождает перевод слева направо и на каждом шаге может выбирать релевантные исходные позиции через cross-attention. Основное вычислительное ограничение

связано не с последовательностью шагов, как у LSTM, а с матрицей попарных attention-связей: при длине последовательности n self-attention требует порядка n^2 сравнений.

Mamba Encoder-Decoder гибрид с Attention. Гибридная Mamba Encoder-Decoder архитектура проверяет, можно ли заменить внутреннее self-attention-моделирование последовательности блоками Mamba, сохранив при этом encoder-decoder способ передачи информации между языками. Encoder формирует представления исходного предложения с помощью Mamba-блоков. Decoder также использует каузальные Mamba-блоки, поскольку целевая последовательность порождается слева направо.

Полная замена attention в задаче перевода нежелательна: decoder все равно должен сопоставлять строящийся перевод с исходным предложением. Поэтому связь между encoder и decoder сохраняется через cross-attention. В такой конфигурации Mamba отвечает за обработку последовательности внутри encoder и decoder, а attention остается механизмом доступа decoder к исходному тексту. Это отделяет вопрос о линейном моделировании контекста от вопроса о выравнивании исходных и целевых токенов.

Transformer Decoder. Transformer Decoder переносит задачу перевода в decoder-only формулировку. Исходное предложение и целевой текст записываются в одну последовательность, а модель обучается продолжать ее токенами перевода. Вся информация об исходном тексте попадает в модель как префикс, а causal self-attention разрешает каждому целевому токenu обращаться к этому префиксу и уже сгенерированной части перевода.

Такой вариант ближе к языковым моделям: одна и та же архитектура описывает перевод как продолжение текста. Цена этой универсальности — отсутствие отдельного encoder и cross-attention. Модель не получает специального блока, который хранит исходное предложение отдельно от целевого префикса, поэтому соответствие между фрагментами исходного текста и перевода должно возникать внутри общего каузального контекста.

Mamba Decoder. Mamba Decoder использует ту же decoder-only постановку, но вместо causal self-attention применяет каузальные Mamba-блоки. Исходное

предложение остается префиксом, перевод строится авторегрессионно, а функция потерь считается по целевым токенам.

Такой вариант интересен как наиболее радикальная замена attention: в нем нет ни encoder, ни cross-attention, ни self-attention между всеми парами токенов. Вычислительная сложность по длине последовательности становится линейной, но возрастает нагрузка на внутреннее состояние Mamba. Для перевода это особенно важно, поскольку модель должна не просто продолжить текст, а сохранить в состоянии сведения об исходном предложении до тех пор, пока они не будут использованы при генерации соответствующих целевых фрагментов.

3 ЭКСПЕРИМЕНТАЛЬНОЕ СРАВНЕНИЕ NMT ПОДХОДОВ

3.1 Подготовка данных

В экспериментах со всеми моделями рассматривается только русско-английская языковая пара. Все модели обучаются переводить предложения между русским и английским языками, поэтому подготовка корпуса направлена не на построение многоязычной системы, а на получение достаточно качественных параллельных пар для одного направления перевода. Для сравнения подходов используются три варианта набора данных разного размера, чтобы анализировать масштабируемость моделей.

Первый набор данных включает OPUS-100 [22] и Yandex English-Russian Parallel Corpus [23]. Он используется для подбора гиперпараметров, так как его размер позволяет проводить несколько серий обучения без чрезмерных вычислительных затрат. Второй набор данных дополнительно включает ParaCrawl [24]. Он применяется для анализа масштабируемости моделей при увеличении объема обучающих данных. Третий набор данных строится на основе OPUS-100, Yandex Corpus, ParaCrawl и United Nations Parallel Corpus [25]. Он используется для обучения и валидации лучшего из рассмотренных подходов.

Для всех трех вариантов корпуса выполняется одинаковая предварительная обработка. На первом этапе удаляются пары с сильным расхождением длины предложений. Пусть l_{ru} и l_{en} — длины русского и английского предложений после предварительной токенизации. Пара отбрасывается, если выполняется условие

$$\frac{|l_{ru} - l_{en}|}{\max(l_{ru}, l_{en})} > 0,3.$$

Фильтр удаляет примеры, в которых одно предложение содержит только часть перевода другого предложения, лишний фрагмент текста или технический мусор. При этом порог не должен быть слишком жестким, поскольку естественная длина русского и английского переводов может различаться.

Следующим этапом выполняется фильтрация по языку. Для этого используется идея language identification, то есть автоматического определения языка

текста. В общем виде такую задачу можно описать как выбор языка с максимальной апостериорной вероятностью:

$$\hat{c}(s) = \arg \max_{c \in C} p(c \mid s),$$

где s — входное предложение, C — множество возможных языков, а c — один из языковых классов. На практике такие модели часто используют символьные признаки и статистику коротких фрагментов текста, поскольку распределения символов и символьных n -грамм хорошо различают языки даже на относительно коротких предложениях [26]. Пара удаляется, если для русской стороны определяется не русский язык или для английской стороны определяется не английский язык.

После этого для каждой пары вычисляется reference-free версия COMET, то есть оценка качества без эталонного перевода. В такой постановке модель получает только два предложения пары и оценивает, насколько одно из них похоже на корректный перевод другого. Такую постановку обычно относят к quality estimation, поскольку качество оценивается без доступа к reference-предложению [27]. Если обозначить русское предложение через x , английское предложение через y , а модель оценки через f_ϕ , то вычисляемый score можно записать как

$$q = f_\phi(x, y).$$

После вычисления оценки остаются только пары, для которых выполняется условие $q > 0,7$. Этот шаг удаляет часть предложений, которые прошли простые фильтры длины и языка, но все еще плохо соответствуют друг другу как переводная пара.

Дополнительно применяется аугментация с помощью LLM-модели. Она используется не для свободного перефразирования корпуса, а для исправления локальных ошибок в уже отобранных парах. К таким ошибкам относятся остатки разметки, технические символы, незначительные нарушения пунктуации, а также небольшие несоответствия по длине, когда одно предложение содержит начало или конец фрагмента, отсутствующий в другом предложении. Этот этап повышает однородность корпуса без изменения основной семантики переводной пары.

После всех этапов подготовки итоговые размеры наборов данных пред-

ставлены в таблице 1.

Таблица 1 – Размеры подготовленных наборов данных

№	Состав корпуса	Размер
1	OPUS-100 и Yandex Corpus	1,9 млн пар
2	OPUS-100, Yandex Corpus и ParaCrawl	4,7 млн пар
3	OPUS-100, Yandex Corpus, ParaCrawl и United Nations Parallel Corpus	7,2 млн пар

3.2 Среда экспериментов и гиперпараметры

В качестве языка реализации используется Python, а модели строятся с помощью фреймворка `torch` [28]. PyTorch удобен для экспериментальной работы с нейронными сетями: в нем можно явно описывать архитектуры, контролировать цикл обучения и использовать аппаратное ускорение для матричных операций. Кроме того, этот фреймворк широко применяется в современных исследованиях по машинному обучению. Для корректного сравнения подходов используется единый обучающий цикл: различаются архитектуры моделей, но не базовая схема оптимизации и вычисления функции потерь.

Модели валидируются в document-level схеме перевода: весь текст параллельного корпуса подается в модель. Качество полученного перевода оценивается по ранее обозначенным в работе метрикам: `bleu`, `chrF` и `comet`.

Для оптимизации используется AdamW [17]. AdamW представляет собой модификацию Adam, в которой `weight decay` отделен от градиента основной функции потерь. Если g_t обозначает градиент на шаге t , то Adam оценивает экспоненциальные скользящие средние первого и второго моментов:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t, \quad v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2.$$

Параметры β_1 и β_2 задают скорость накопления этих средних. После коррекции смещения параметры модели обновляются с учетом `learning rate` η_t и отдельного

коэффициента weight decay λ :

$$w_{t+1} = (1 - \eta_t \lambda) w_t - \eta_t \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \varepsilon}.$$

В качестве основной функции потерь используется cross entropy loss. Для авторегрессионной модели перевода она имеет вид:

$$\mathcal{L}_{CE} = - \sum_{t=1}^n \log p_{\theta}(y_t \mid y_{<t}, x),$$

где x — исходное предложение, y_t — целевой токен на шаге t , а $y_{<t}$ — уже известный префикс перевода. При использовании label smoothing вместо one-hot распределения применяется сглаженное целевое распределение, что уменьшает чрезмерную уверенность модели в одном токене. В экспериментах регуляризация включает три независимых механизма: dropout, сглаживание целевых меток и ограничение величины весов через оптимизатор.

В современных практиках обучения моделей используют не постоянный learning rate. Закон, по которому меняется learning rate, называется “расписанием”. Выбором расписания, как правило, устраняют эмпирически установленные недостатки различных стадий обучения модели. Например, на самом старте обучения, когда веса модели “далеко” от оптимальных, градиент слишком велик, и оптимизация модели слишком нестабильна из-за проблемы “градиентного взрыва”. В таком случае лучше иметь меньший learning rate. Поскольку увеличение размера модели обычно усиливает нестабильность во время обучения, использование расписания или уменьшение learning rate становится обязательным. При этом, после нестабильного старта слишком малый learning rate иметь нежелательно, так как это замедляет процесс обучения. На поздних стадиях обучения, когда модель совершила несколько “эпох”, происходит более детальный подбор параметров, необходимо совершать более точные шаги, а значит необходим меньший learning rate.

Одна из распространенных схем расписания learning rate решает упомянутые выше проблемы. На первых шагах обучения применяется warmup: learning rate по линейному закону увеличивается от малого заданного значения до базового уровня. Идея warmup широко используется в обучении Transformer-моделей [3]. После завершения warmup learning rate уменьшается по косинус-

ному закону [29], известному как “cosine annealing”:

$$\eta_t = \eta_{min} + \frac{1}{2}(\eta_{max} - \eta_{min}) \left(1 + \cos \frac{\pi(t - T_w)}{T - T_w} \right), \quad t \geq T_w,$$

где T_w — число warmup-steps, T — общее число шагов обучения, η_{max} — максимальное значение learning rate, а η_{min} — минимальное значение learning rate. В отличие от ступенчатого уменьшения learning rate, cosine annealing не создает резких скачков в динамике оптимизации. Описанное выше расписание требует задания двух дополнительных гиперпараметров: начальный learning rate и число warmup шагов.

Гиперпараметром можно назвать и размер словаря. Несмотря на то, что изменение размера словаря фактически меняет веса модели (поскольку меняется размер embedding слоя), сама вычислительная схема модели остается такой же. Слишком большой словарь может излишне увеличить размер модели, что стоит учитывать при подборе этого параметра.

В каждой из рассматриваемых архитектур есть параметры, задающие размер слоев. Под скрытым слоем здесь понимается любой слой между входом нейронной сети и выходом. Размерность входа и выхода модели, как правило, фиксированная, а вот размерность векторов между скрытыми слоями можно задавать, масштабируя слой. Обычно между скрытыми слоями оставляют одну и ту же размерность. Сравнение моделей с разным hidden size показывает, как модель масштабируется в ширину.

Наряду с масштабированием “в ширину” можно изменять число одинаковых скрытых слоев, сравнивая масштабируемость модели “в глубину”. Изменение глубины может приводить к более существенным изменениям в процессе обучения. Одна из причин связана с распространением градиента через последовательность слоев. Если выход слоя записать как $h_l = f_l(h_{l-1})$, то градиент по раннему представлению вычисляется по цепному правилу:

$$\frac{\partial \mathcal{L}}{\partial h_0} = \frac{\partial \mathcal{L}}{\partial h_L} \prod_{l=1}^L \frac{\partial h_l}{\partial h_{l-1}}.$$

При увеличении числа слоев произведение якобианов становится длиннее. Если нормы множителей в среднем меньше единицы, градиент затухает и нижние слои обновляются слабо; если больше единицы, возникает градиентный взрыв

и шаги оптимизации становятся нестабильными. Поэтому увеличение глубины влияет не только на число параметров, но и на устойчивость обучения.

Для ускорения вычислений при обучении применялся `automatic mixed precision`. В качестве пониженной точности используется формат `bfloat16`, а не `float16`. Основная причина — сохранение 8-битной экспоненты, как у `float32`; поэтому `bfloat16` имеет близкий к `float32` диапазон представимых значений [30]. У `float16` диапазон уже, поэтому при обучении глубоких моделей чаще возникают переполнения и исчезновение малых градиентов, а также может потребоваться дополнительное масштабирование `loss`-функции. Формат `bfloat16` имеет меньшую точность мантииссы, но для обучения нейронных сетей это обычно менее критично, чем устойчивый диапазон значений. Поэтому `bfloat16` уменьшает потребление памяти и ускоряет матричные операции без существенного изменения гиперпараметров обучения.

3.3 Схема подбора гиперпараметров

Полный перебор гиперпараметров для всех рассматриваемых архитектур практически невозможен. Даже при небольшом числе значений для `learning rate`, размера `batch`, `hidden size`, числа слоев, параметров регуляризации и расписания обучения количество запусков быстро становится слишком большим. При этом каждый запуск требует обучения модели на корпусе значительного размера, поэтому схема экспериментов должна учитывать ограничение вычислительных ресурсов.

В работе используется допущение, что модели разных архитектур имеют часть общих свойств обучения. Например, слишком большой `learning rate` может приводить к нестабильной оптимизации как для `Transformer`, так и для `Mamba`, а чрезмерный `dropout` может ухудшать сходимость независимо от конкретного блока. Поэтому глобальные параметры, подобранные на одной модели, рассматриваются как приближенно подходящие и для других моделей. Такое допущение не означает, что найденные значения являются строго оптимальными для каждой архитектуры, но позволяет сократить число экспериментов до практически выполнимого набора.

Второе допущение состоит в том, что большинство гиперпараметров можно подбирать последовательно. В этом случае сначала фиксируется базовый

вая конфигурация, затем изменяется один параметр или небольшая группа связанных параметров, после чего выбранное значение используется в следующих запусках. Такой подход не гарантирует нахождение глобального оптимума, однако позволяет оценить влияние отдельных решений без полного перебора всех комбинаций.

Некоторые параметры нельзя считать независимыми. Learning rate связан с размером batch: при изменении числа примеров в одном шаге меняется шум градиента и масштаб обновлений. Параметры warmup также рассматриваются совместно, поскольку число warmup-steps и начальное значение learning rate определяют одну и ту же начальную фазу оптимизации. Hidden size, число слоев и размер обучающего набора связаны через емкость модели: увеличение глубины или ширины без достаточного объема данных может усиливать переобучение, а слишком малая модель может не использовать преимущества большего корпуса. Поэтому такие параметры подбираются не по отдельности, а как связанные группы.

Значения loss корректно сравнивать только для запусков с одинаковыми конфигурациями словарей и одинаковым значением label smoothing. Размер и состав словаря напрямую определяют пространство классов в cross entropy loss, а label smoothing изменяет целевое распределение вероятностей. Поэтому при различии этих параметров loss отражает не только качество модели, но и изменение самой функции потерь. На практике увеличение label smoothing искусственно завышает значение loss, даже если качество перевода по внешним метрикам не ухудшается.

3.4 Эксперименты с токенизатором

При обучении токенизаторов использовалось предварительное разбиение, в котором цифры и знаки препинания отделялись от соседних буквенных фрагментов, так как числа и пунктуация часто должны переноситься в перевод почти без изменения. Это было подтверждено экспериментально: добавление такой предобработки улучшило точность переноса чисел и знаков препинания в переведенный текст.

Размер словаря выбирался по моменту, после которого увеличение числа подслов почти не отражалось на внешних метриках. Для небольшого корпу-

са словарь больше 36 тыс. токенов оказывался избыточным: новые элементы встречались редко, но сразу увеличивали embedding-слой и выходную проекцию. На среднем наборе данных рабочим оказался уровень около 48 тыс. токенов. Для большого корпуса граница смещалась примерно к 64 тыс.; редкие под слова в этом случае получают больше наблюдений, и разрежение статистики становится менее заметным.

Для Transformer encoder-decoder схемы был сделан дополнительный запуск с BPE. Обучались две двуязычные модели со словарем 48 тыс. токенов. Конфигурация базовой encoder-decoder модели приведена в приложении А. Обучение шло одну эпоху на датасете среднего размера; архитектура, optimizer и расписание learning rate не менялись. В таблицу не включен loss, поскольку при другом словаре меняется число классов и распределение частот токенов.

Таблица 2 – Сравнение unigram и BPE для базовой Transformer encoder-decoder модели

Токенизатор	BLEU	chrF	COMET
unigram	20,78	51,04	0,7385
BPE	21,07	51,47	0,7383

BPE дал +0,29 BLEU и +0,43 chrF при практически совпадающем COMET. Такой результат не распространился на все архитектуры, но для Transformer encoder-decoder BPE оказался лучшим вариантом. В межархитектурных сравнениях далее будет использоваться unigram, так как он дает более стабильные результаты в среднем по альтернативным (LSTM, Mamba) архитектурам.

3.5 Эксперименты с параметрами обучения

В следующих экспериментах использовалась модель, уже заданная в экспериментах с токенизатором (“небольшой” transformer encoder-decoder). Эксперименты также проводились на протяжении одной эпохи датасета средней конфигурации.

Расписание learning rate. На основе нескольких первых экспериментов, до полноценного подбора гиперпараметров, был выбран единый формат расписа-

ния learning rate - linear warmup + cosine annealing с фиксированными гиперпараметрами. В warmup фазе было решено использовать learning rate равный 1% от базового значения, на протяжении одной пятой эпохи. Минимальное значение при annealing выставлено равным 10% от базового learning rate. Примерный вид использованного расписания показан на рисунке 5, фрагмент создания scheduler в PyTorch приведен в приложении А.

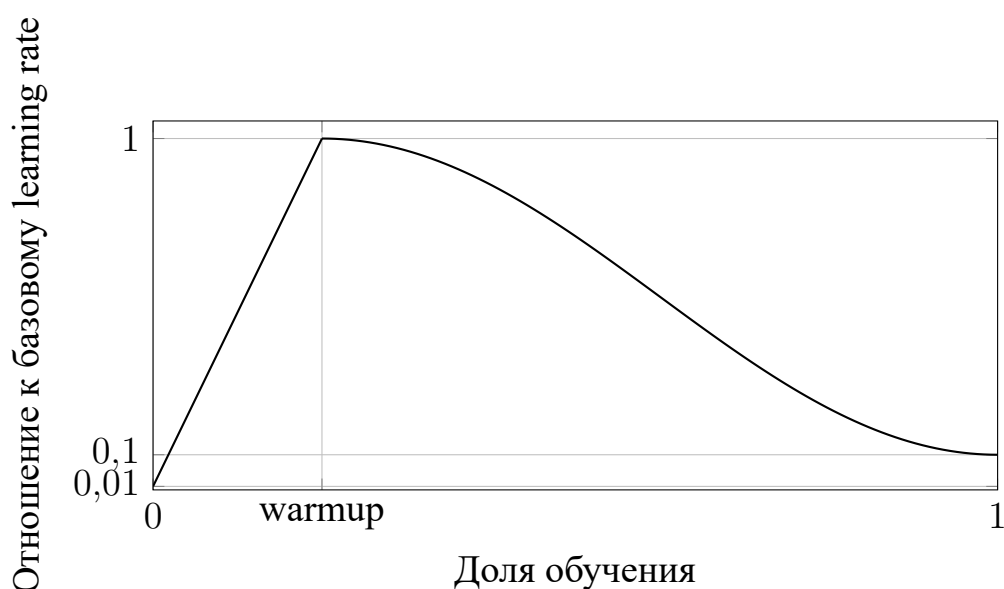


Рисунок 5 – Схематичный вид расписания learning rate

После фиксации формы расписания сравнивались несколько базовых значений learning rate. Основное сравнение выполнялось при достаточно большом batch size, равном 100. Слишком малое значение $3 \cdot 10^{-5}$ приводило к медленному обучению: за одну эпоху модель оставалась на низком уровне качества. Значения $3 \cdot 10^{-4}$ и $3 \cdot 10^{-3}$ уже давали рабочую динамику, при этом в данном коротком запуске более высокий learning rate оказался лучше по всем внешним метрикам. На более длинных запусках была замечена закономерность при использовании слишком больших learning rate: функция потерь, несмотря на быстрое уменьшение в начале, также быстрее выходила на плато, из-за чего конечный результат был хуже.

Таблица 3 – Влияние базового learning rate на качество перевода

Learning rate	BLEU	chrF	COMET	Loss
$3 \cdot 10^{-5}$	6,35	30,49	0,4566	4,312
$3 \cdot 10^{-4}$	20,96	51,26	0,7384	3,066
$3 \cdot 10^{-3}$	22,97	52,93	0,7658	2,919

При уменьшении batch size соотношение изменилось: значение $3 \cdot 10^{-4}$ оказалось устойчивее, чем $3 \cdot 10^{-3}$. Дополнительное сравнение приведено в таблице 4.

Таблица 4 – Влияние learning rate при меньшем batch size

Learning rate	BLEU	chrF	COMET	Loss
$3 \cdot 10^{-5}$	15,84	45,72	0,6731	3,487
$3 \cdot 10^{-4}$	21,64	52,04	0,7506	3,018
$3 \cdot 10^{-3}$	20,88	50,92	0,7337	3,141

Такое поведение можно объяснить через стохастическую природу mini-batch обучения. Градиент, вычисленный по batch, является шумной оценкой полного градиента, причем дисперсия этой оценки уменьшается при росте batch size. Если batch size уменьшается, то при том же learning rate возрастает относительная величина случайной составляющей шага оптимизации. В работах по связи learning rate и batch size этот эффект описывается через noise scale, который для малых batch относительно размера датасета приблизительно пропорционален отношению learning rate к batch size [31]. Поэтому уменьшение batch size может требовать уменьшения learning rate: в противном случае шаги становятся более шумными и модель быстрее попадает в менее устойчивую область оптимизации.

Параметры AdamW. Для AdamW отдельно проверялись коэффициенты экспоненциального сглаживания первого и второго моментов градиента. При изменении β_1 значение β_2 оставалось фиксированным, а при изменении β_2 фиксировалось β_1 .

Таблица 5 – Влияние β_1 в AdamW на качество перевода

β_1	BLEU	chrF	COMET	Loss
0,85	21,48	51,61	0,7409	3,063
0,90	20,82	50,87	0,7363	3,076
0,95	21,06	51,29	0,7396	3,057

Различия между значениями β_1 получились небольшими. Наилучший BLEU, chrF и COMET в этой группе дал вариант 0,85, но преимущество над 0,95 не выглядит большим. Поэтому β_1 можно считать, вероятно, менее чувствительным параметром, чем learning rate.

Таблица 6 – Влияние β_2 в AdamW на качество перевода

β_2	BLEU	chrF	COMET	Loss
0,95	18,05	47,72	0,6820	3,342
0,98	19,97	49,84	0,7166	3,180
0,999	21,20	51,27	0,7383	3,064

Для β_2 зависимость выражена сильнее: увеличение коэффициента до 0,999 улучшило все внешние метрики и одновременно снизило loss. Вероятная причина состоит в более стабильной оценке второго момента градиента. Результат интересен в первую очередь тем, что большинство современных больших языковых моделей обучается с $\beta_2 = 0,95$, но это значение не является оптимальным в проведенных экспериментах.

Weight decay. Weight decay применялся не ко всем параметрам модели: из него исключались bias-параметры, embedding-слои и normalization-слои [32]. Embedding матрицу можно рассматривать как таблицу значений, сопоставляющую одному токenu его векторное представление. Если в процессе градиентного спуска применить decay к этой таблице, векторы, соответствующие редким токенам, будут задействованы редко, и градиент по ним “протекать” не будет. Таким образом, decay будет просто “тянуть” векторы редких токенов к нулю, и

только иногда они будут обновляться:

$$e_{i,t+1} = \begin{cases} (1 - \eta_t \lambda) e_{i,t} - \eta_t u_{i,t}, & i \in B_t, \\ (1 - \eta_t \lambda) e_{i,t}, & i \notin B_t, \end{cases}$$

где e_i – embedding-вектор токена i , B_t – множество токенов в текущем mini-batch, η_t – learning rate, λ – weight decay, а $u_{i,t}$ – градиентное обновление для этого embedding-вектора. Если токен i редкий и не встречается в течение K шагов подряд, то для него $i \notin B_t$ на каждом из этих шагов. В таком случае embedding-вектор не получает содержательного градиентного обновления и изменяется только за счет decay:

$$e_{i,t+K} = e_{i,t} \prod_{s=t}^{t+K-1} (1 - \eta_s \lambda) \approx e_{i,t} \exp \left(-\lambda \sum_{s=t}^{t+K-1} \eta_s \right).$$

При положительных η_s и λ множитель перед $e_{i,t}$ становится меньше единицы, поэтому норма embedding-вектора постепенно уменьшается. Когда редкий токен снова появляется в batch, его вектор уже может быть ближе к нулю, и одно градиентное обновление потенциально будет снова обучать модель тому, что она уже знала. Для слоев нормализации weight decay форсирует модель работать с низкочастотными сигналами, так как значение scale параметра нормализации стягивается к нулю. Аналогичное объяснение можно применить и к bias параметрам линейных слоев – decay тянет значение сдвига к нулю, убирая всю пользу bias параметров в целом. Фрагмент реализации такого разделения параметров на уровне оптимизатора приведен в приложении А.

Таблица 7 – Влияние weight decay на качество перевода

Weight decay	BLEU	chrF	COMET	Loss
0,005	20,82	50,88	0,7364	3,078
0,01	20,85	50,94	0,7371	3,071
0,1	21,03	51,18	0,7390	3,055

Различия между значениями weight decay получились небольшими. Наилучшие значения внешних метрик дал вариант 0,1, однако отрыв от 0,01 и 0,005

остается малым. Поэтому этот параметр в проверенном диапазоне выглядит менее чувствительным, чем learning rate или β_2 .

Dropout. Dropout проверялся как основной параметр регуляризации модели. В коротком запуске на датасете среднего размера более сильное зануление активаций не дало преимущества: качество постепенно снижалось при переходе от 0,05 к 0,2.

Таблица 8 – Влияние dropout на качество перевода

Dropout	BLEU	chrF	COMET	Loss
0,05	21,40	51,21	0,7380	3,058
0,10	20,69	50,91	0,7348	3,071
0,20	20,10	50,51	0,7311	3,093

Однако модели большего размера (8 слоев encoder и decoder) без увеличения размера датасета лучше показывали себя на конфигурации с $dropout = 0,1$.

Таблица 9 – Влияние dropout для модели большего размера

Dropout	BLEU	chrF	COMET	Loss
0,05	23,74	53,31	0,7724	2,842
0,10	24,18	53,86	0,7791	2,811
0,20	22,96	52,74	0,7638	2,889

Эксперимент подтверждает эмпирическое суждение о том, что модели большего размера сильнее подвержены переобучению. Этот факт учитывался при проведении финальных экспериментов.

Label smoothing. Label smoothing проверялся на двух одинаковых Transformer encoder-decoder конфигурациях; менялось только значение коэффициента сглаживания. Конфигурация модели приведена в приложении А. В первом запуске использовалось 0,05, во втором — 0,1.

При сглаживании часть вероятностной массы переносится с правильного токена на остальные токены словаря. Loss между такими запусками напрямую

не сравнивался: при изменении label smoothing меняется целевое распределение, а значит и сама численная шкала функции потерь. Поэтому ниже приведены только внешние метрики качества перевода.

Таблица 10 – Влияние label smoothing на качество перевода

Label smoothing	BLEU	chrF	COMET
0,05	31,68	60,19	0,8474
0,1	32,33	60,61	0,8497

При 0,1 все три внешние метрики оказались выше: BLEU на 0,65, chrF на 0,42, COMET на 0,0023. Разница невелика, но направлена одинаково по всем метрикам, поэтому 0,1 был взят как рабочее значение для последующих запусков.

GRPO-дообучение. Для encoder-decoder модели был выполнен короткий этап GRPO-дообучения. За основу бралась уже обученная Transformer encoder-decoder конфигурация; архитектура при этом не менялась. Конфигурация модели и параметры GRPO-дообучения приведены в приложении А. Reward задавался как взвешенная сумма нормированных автоматических метрик качества перевода:

$$R = \frac{1}{3} \cdot \frac{\text{chrF}}{60} + \frac{2}{3} \cdot \frac{\text{COMET}}{0,845}.$$

COMET получил больший вес, поскольку reward должен был учитывать не только поверхностные совпадения с reference, но и смысловую близость перевода. chrF оставлен как более простой и устойчивый символьный сигнал. BLEU в reward не включался: для коротких предложений он часто дает шумную оценку и может поощрять слишком буквальное совпадение n -грамм.

Для каждого исходного предложения генерировалась группа из четырех вариантов перевода. Из-за дополнительной генерации (семплирования) GRPO-шаг значительно дороже обычного шага supervised-обучения: модель не только считает loss по готовому target, но и сначала порождает несколько кандидатов. Поэтому за время дообучения было обработано только около 50 тыс. обучающих примеров.

Сравнение базовой модели и модели после GRPO-дообучения приведено

в таблице 11.

Таблица 11 – Сравнение базовой модели и GRPO-дообучения

Модель	BLEU	chrF	COMET
Базовая модель	32,12	60,37	0,8497
После GRPO-дообучения	32,12	60,71	0,8527

После GRPO-дообучения BLEU остался на том же уровне, chrF вырос на 0,34, COMET – на 0,0030. Улучшение произошло только в тех метриках, что явно входили в формулу reward, поэтому результат скорее похож на подстройку модели под выбранную награду, чем на общее улучшение перевода. При этом затраченное время было сопоставимо с заметной частью supervised-обучения. Таким образом более выгодным выглядит улучшение данных и supervised этапа; для GRPO, вероятно, нужна более экономичная схема генерации или другая reward-функция. Ожидания от этого этапа обучения были выше и не оправдались.

3.6 Эксперименты с архитектурами моделей

Архитектуры сравнивались на схожем размере по числу параметров. Точного равенства добиться сложно: у encoder-decoder и decoder-only моделей по-разному распределяются параметры между embedding, блоками последовательности и выходной проекцией. Тем не менее большинство запусков попадало в диапазон примерно от 90 до 130 млн обучаемых параметров. Для обучения использовался датасет среднего размера с подобранными ранее параметрами обучения. Результаты обучения на основе 10 полных эпох представлены в таблице 12.

Таблица 12 – Сравнение архитектур моделей NMT

Архитектура	Схема	BLEU	chrF	COMET
LSTM Encoder-Decoder	encoder-decoder, LSTM с attention	26,18	55,9	0,813
Transformer Encoder-Decoder	encoder-decoder, self-attention и cross-attention	29,34	58,8	0,836
Mamba Encoder-Decoder гибрида	encoder-decoder, Mamba-блоки и cross-attention	28,94	58,6	0,833
Transformer Decoder	decoder-only, causal self-attention	27,87	57,3	0,824
Mamba Decoder	decoder-only, каузальные Mamba-блоки	18,66	49,4	0,706

LSTM Encoder-Decoder уступил Transformer-вариантам по всем трем метрикам; разрыв по BLEU превышает 3 пункта. Результат в целом ожидаем – Transformer архитектура со временем показала свое преимущество над рекуррентными подходами. Однако результат LSTM модели все еще удовлетворительный: модель смогла выдавать адекватный и различимый перевод на текстах малого размера.

Среди encoder-decoder запусков Transformer и Mamba-гибрид оказались близки. Transformer выше на 0,40 BLEU, 0,2 chrF и 0,003 COMET, то есть разрыв небольшой относительно общей разницы между архитектурными группами. Такой результат разумнее трактовать осторожно: Mamba-блоки в этой конфигурации не разрушили качество при замене части self-attention, но и не дали заметного преимущества. Возможно, решающим для обеих моделей здесь был не столько сам блок внутри encoder и decoder, сколько сохраненный cross-attention между исходной и целевой последовательностями.

Decoder-only модели на таком масштабе размеров оказались хуже encoder-decoder вариантов. Поскольку Decoder модели довольно просты по своему устройству и лучше всего подходят для общей задачи языкового моделирования, их можно обучить на большом корпусе любых текстов в целом, в совсем другом масштабе параметров, чем исследуемые в работе. Такая модель, содержащая общие знания о языке, вероятно сможет ценой большей вычислительно нагрузки показать результат лучший, чем encoder-decoder модель. Именно

по этой причине современные LLM все чаще используются для задачи NMT — их проще масштабировать и дообучать под конкретный домен перевода. Однако в условиях ограниченных вычислительных ресурсов можно заключить, что encoder-decoder модели показывают себя лучше.

Mamba Decoder получил самый низкий результат. Здесь одновременно отсутствуют отдельный encoder, cross-attention и попарный self-attention между всеми позициями. Поэтому модель должна удерживать сведения об исходном предложении во внутреннем состоянии каузального блока. Для языкового продолжения текста такой режим может быть естественным, но для перевода он предъявляет более жесткие требования к сохранению деталей исходной последовательности, что на таком размере модели скорее всего, невозможно.

В качестве оптимальной была выбрана Transformer Encoder-Decoder архитектура. В сравнении encoder-decoder Transformer дал лучший набор метрик и оставил понятные параметры для последующей настройки — позиционное представление, размер словаря, регуляризацию, глубину и hidden size. Ключевой фрагмент реализации модели приведен в приложении А.

3.7 Эксперименты с параметрами моделей

Позиционное представление. Для оценки влияния способа задания позиции сравнивались две Transformer encoder-decoder модели сопоставимого размера. В первой модели использовалось absolute positional encoding, во второй — rotary positional embeddings. Остальные основные архитектурные параметры совпадали: число слоев encoder и decoder, hidden size, число attention-голов, размер feed-forward блока и dropout. Конфигурации моделей приведены в приложении А. Для обучения был использован датасет большого размера.

Сравнение выполнялось по лучшим значениям метрик на validation-наборе, было совершено 10 полных эпох обучения. Результаты представлены в таблице 13.

Таблица 13 – Сравнение positional encoding и RoPE

Позиционное представление	BLEU	chrF	COMET	Loss
Absolute PE	31,22	59,78	0,8450	1,931
RoPE	31,59	59,84	0,8484	1,914

Разница в пользу RoPE небольшая: сильнее всего изменились BLEU и COMET, chrF почти не сдвинулся. Тем не менее направление изменения одинаковое для всех метрик, поэтому RoPE был оставлен в следующих Transformer encoder-decoder запусках. Возможное объяснение связано с тем, что позиция в RoPE входит не как добавка к embedding, а непосредственно в скалярные произведения query и key, от которых зависят attention-веса.

Направление перевода. Также сравнивались две encoder-decoder модели в однонаправленной и двуязычной постановках. В однонаправленной постановке модель обучалась переводить только из русского языка в английский. В двуязычной постановке одна модель обучалась обоим направлениям перевода с указанием целевого языка во входной последовательности. Для сравнения использовались модели с одинаковыми основными архитектурными параметрами: Transformer encoder-decoder с RoPE, 8 слоев encoder, 8 слоев decoder, hidden size 768, 12 attention-голов, feed-forward multiplier 3 и dropout 0,05.

Результаты сравнения приведены в таблице 14.

Таблица 14 – Сравнение однонаправленной и двуязычной encoder-decoder постановки

Постановка обучения	BLEU	chrF	COMET	Loss
Однонаправленная	31,59	59,84	0,8484	1,914
Двуязычная	32,05	60,35	0,8493	1,819

Двуязычная постановка в этом сравнении оказалась выше по BLEU, chrF и COMET, а validation loss также снизился. Возможная причина — более полное использование параллельного корпуса: обе стороны выступают и как источник, и как целевой текст. Параметры encoder и decoder при этом остаются общими,

так что модель получает больше разнообразных обучающих примеров без изменения архитектуры.

Масштабирование размера. В рамках поставленных ранее экспериментов была получена ожидаемая связь размера модели и размера обучающего набора данных. Увеличение числа слоев и hidden size улучшало качество только до тех пор, пока обучающий корпус давал достаточно примеров для настройки дополнительных параметров. После этого более крупная модель начинала обучаться медленнее, а прирост внешних метрик становился небольшим или исчезал.

Ограничение также зависело от архитектуры. LSTM encoder-decoder модель было трудно масштабировать выше 8 слоев encoder и decoder: обучение становилось заметно медленнее, а в отдельных запусках появлялась нестабильность, похожая на градиентный взрыв.

Для Transformer encoder-decoder и гибридной Mamba encoder-decoder архитектур масштабирование выглядело более устойчивым, удалось установить оптимальные варианты конфигурации моделей в зависимости от объема обучающих данных, приведенные в таблице 15.

Таблица 15 – Практический предел размера Transformer/Mamba encoder-decoder модели

Размер датасета	Число слоев encoder/decoder	Hidden size
Малый	6/6	512
Средний	8/8	768
Большой	12/12	1024

При дальнейшем увеличении размера модель получала слишком большую емкость относительно объема данных, поэтому качество на validation-наборе росло слабо. Для последующего итогового обучения был выбран промежуточный вариант Transformer encoder-decoder: 8 слоев encoder, 8 слоев decoder и hidden size 768. Он соответствовал датасету среднего и большого размера, но не требовал столь больших вычислительных затрат, как конфигурация 12/12 с hidden size 1024.

3.8 Обучение лучшей конфигурации

После подбора гиперпараметров была обучена итоговая двуязычная модель перевода. В качестве архитектуры используется Transformer encoder-decoder с rotary positional embeddings. Модель содержит около 230 млн параметров и обучается на наибольшем подготовленном наборе данных в течение 10 эпох. Основные параметры итогового обучения приведены в таблице 16.

Таблица 16 – Гиперпараметры итоговой модели

Параметр	Значение
Архитектура	Transformer encoder-decoder с RoPE
Число параметров	230 млн
Языковая постановка	двуязычная модель ru–en
Число слоев encoder и decoder	8 и 8
Hidden size	768
Число attention-голов	12
Число эпох	10
Batch size	48
Learning rate	0,0003
Минимальный learning rate	0,00001
Начальный learning rate warmup	$0,01 \cdot lr$
Параметры AdamW	$\beta_1 = 0,9, \beta_2 = 0,999$
Label smoothing	0,1
Dropout	0,1
Weight decay	0,05

Во время обучения после каждой десятой части эпохи вычислялись значения validation loss, BLEU, chrF и COMET. Динамика BLEU, chrF и COMET на validation-наборе показана на рисунках 6–8.

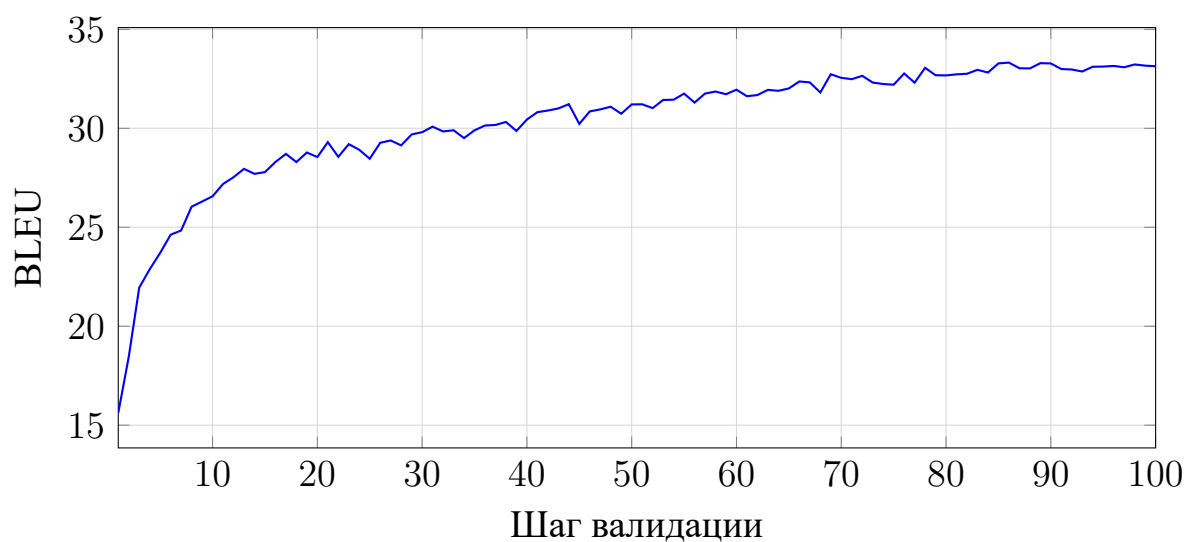


Рисунок 6 – Динамика BLEU итоговой модели на validation-наборе

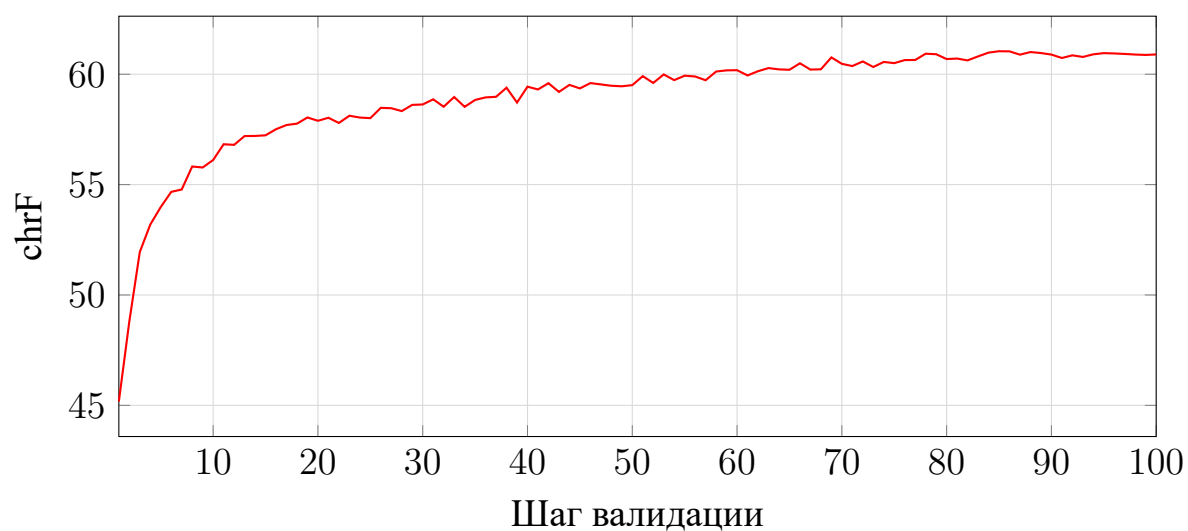


Рисунок 7 – Динамика chrF итоговой модели на validation-наборе

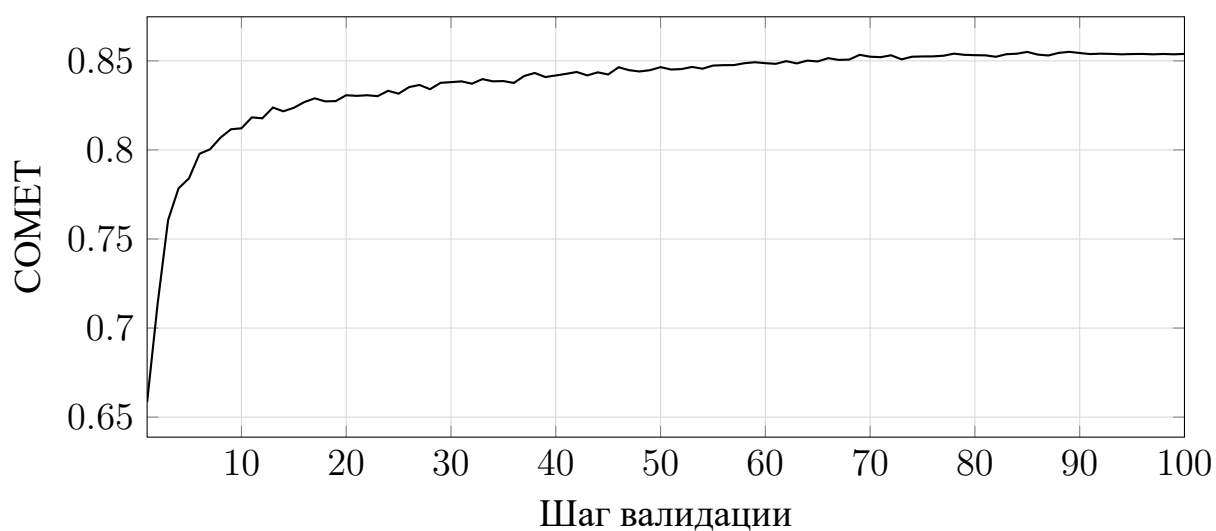


Рисунок 8 – Динамика COMET итоговой модели на validation-наборе

В начале обучения BLEU, chrF и COMET растут быстрее всего; затем графики переходят к более плавному участку. К концу обучения метрики в основном колеблются около достигнутого уровня. Поэтому простое добавление новых эпох без изменения данных, регуляризации или размера модели, скорее всего, дало бы меньший эффект, чем изменения на более ранних этапах экспериментов.

Итоговая оценка выполнялась на тестовом наборе, который не использовался при подборе гиперпараметров. На test-наборе модель получила следующие значения: **BLEU** – 32,25, **chrF** – 60,34, **COMET** – 0,8519.

Примеры переводов итоговой модели приведены в таблице 17. В этих примерах модель корректно передает даты, в том числе века, записанные римскими цифрами, и сохраняет основные имена собственные. Пунктуация и числовые фрагменты также в основном перенесены без искажений.

Таблица 17 – Примеры переводов итоговой модели

Исходное предложение	Образец	Перевод модели
Образец Эпоху, в которой происходили данные события, принято называть Высоким Средневековьем, периодом европейской истории, затрагивающим XI, XII и XIII века (1000–1300 гг. н. э.).	The age where the events took place is commonly referred as the High Middle Ages the period of European history in the 11th, 12th, and 13th centuries (AD 1000–1300).	The era in which these events occurred is called the High Middle Ages, a period of European history covering the 11th, 12th, and 13th centuries (1000–1300 AD).

Исходное предложение	Образец	Перевод модели
В 1683 году силы династии Цин (1644–1912) взяли под контроль западные и северные прибрежные районы Тайваня и в 1885 году объявили Тайвань провинцией империи Цин.	In 1683, Qing dynasty (1644–1912) forces take control of Taiwan’s western and northern coastal areas and declared Taiwan as a province of the Qing Empire in 1885.	In 1683, the Qing Dynasty (1644–1912) took control of Taiwan’s western and northern coastal areas and declared Taiwan a province of the Qing Empire in 1885.
В 1994 году этот конфликт привёл к созданию на востоке Молдовы самопровозглашённой Приднестровской республики, которая имеет собственное правительство и валюту, но не признана ни одним государством-членом ООН.	In 1994, this conflict led to the creation of the self-proclaimed Transnistria Republic in eastern Moldova, which has its own government and currency but is not recognised by any UN member country.	In 1994, this conflict led to the creation of a self-proclaimed Transnistrian republic in eastern Moldova, which has its own government and currency, but is not recognized by any UN member state.

ЗАКЛЮЧЕНИЕ

В NMT качество перевода зависит не от одной архитектуры, а от сочетания обучающих данных, способа токенизации, схемы модели и конфигурации оптимизации. Поэтому корректное сравнение архитектур требует сначала зафиксировать факторы, которые иначе становятся скрытыми источниками различий между запусками. Подготовка корпуса и выбор токенизатора задают входное представление и статистику целевых токенов, гиперпараметры обучения определяют устойчивость оптимизации, и только после этого сравнение архитектур становится целесообразно. По этой причине каждый элемент, поддающийся модификации, отдельно анализируется по определенным метрикам качества: BLEU, chrF и COMET, а также по устойчивости обучения и вычислительной стоимости запусков.

Для русско-английского направления были подготовлены три варианта параллельного корпуса объемом 1,9, 4,7 и 7,2 млн пар предложений. Подготовка данных включала фильтрацию по соотношению длин, автоматическое определение языка, оценку качества переводных пар и дополнительную очистку локальных ошибок. Эксперименты с токенизацией показали, что разбиение цифр и знаков препинания на отдельные элементы улучшает перенос числовых и пунктуационных фрагментов, а разумный размер словаря зависит от объема обучающего корпуса. Для небольшой и средней конфигураций избыточное увеличение словаря приводило главным образом к росту embedding-слоя и выходной проекции, тогда как для большего корпуса допустимый размер словаря увеличивался.

Подбор параметров обучения показал, что наибольшее влияние оказывают learning rate, его связь с batch size и параметры адаптивного оптимизатора. При большем batch size модель устойчивее переносила более высокий learning rate, тогда как при уменьшении batch size лучшим становилось меньшее значение learning rate. Это согласуется со стохастической интерпретацией mini-batch обучения: уменьшение batch size увеличивает шум оценки градиента и требует более осторожного шага оптимизации. Для AdamW наиболее значимым из проверенных коэффициентов оказался β_2 : значение 0,999 дало более стабильное обучение, чем меньшие значения. Weight decay, dropout и label smoothing

также влияли на качество, но их оптимальные значения зависели от размера модели и обучающего корпуса.

Сравнение архитектур подтвердило преимущество encoder-decoder постановки в условиях ограниченного размера модели и тренировочного набора данных. Среди моделей сопоставимого размера Transformer encoder-decoder показал лучший результат: 29,34 BLEU, 58,8 chrF и 0,836 COMET. Гибридная Mamba encoder-decoder модель оказалась близка к нему по качеству, что показывает возможность частичной замены self-attention блоками Mamba при сохранении cross-attention между исходной и целевой последовательностями. LSTM encoder-decoder уступил Transformer-модели, но сохранил приемлемое качество на коротких текстах. Decoder-only варианты в проведенных экспериментах оказались менее эффективными, особенно Mamba Decoder, для которого отсутствие отдельного encoder и cross-attention существенно усложняло сохранение информации об исходном предложении.

Эксперименты с масштабированием показали, что увеличение глубины и hidden size полезно только при достаточном объеме данных. LSTM-модели было трудно масштабировать выше 8 слоев encoder и decoder из-за замедления обучения и нестабильности градиентов. Transformer encoder-decoder и гибридная Mamba encoder-decoder архитектуры масштабировались устойчивее: для малого датасета практический предел находился около 6/6 слоев и hidden size 512, для среднего — около 8/8 слоев и hidden size 768, для большого — около 12/12 слоев и hidden size 1024. Поэтому размер модели должен выбираться совместно с размером корпуса, а не рассматриваться как независимый гиперпараметр.

На основе проведенных сравнений была обучена итоговая двуязычная Transformer encoder-decoder модель с RoPE, 8 слоями encoder, 8 слоями decoder, hidden size 768 и приблизительно 230 млн параметров. На независимом test-наборе она достигла 32,25 BLEU, 60,34 chrF и 0,8519 COMET. Примеры перевода показывают, что модель корректно передает основное содержание предложений, сохраняет даты, числовые фрагменты и значительную часть имен собственных. Это подтверждает, что выбранная конфигурация является работоспособной для русско-английского перевода в рамках доступного корпуса и вычислительного бюджета.

Дорогое по вычислениям GRPO-дообучение привело к небольшому ро-

сту chrF и COMET при неизменном BLEU. Поскольку улучшение наблюдалось именно по метрикам, входившим в reward, результат следует трактовать как ограниченную подстройку модели под заданную функцию награды. С учетом высокой стоимости генерации нескольких вариантов перевода для каждого исходного предложения supervised-обучение и улучшение качества данных в данной постановке выглядят более эффективными направлениями повышения качества.

Ограничения исследования связаны с одной языковой парой, автоматической оценкой качества и конечным объемом вычислительных ресурсов. Более полная проверка полученных выводов требует экспериментов на других языковых направлениях, доменах и размерах моделей, а также оценки переводов человеком. Перспективными направлениями дальнейшей работы являются более экономичные схемы reward-based дообучения, а также прямое сравнение специализированных encoder-decoder систем с крупными предобученными decoder-only моделями при сопоставимых вычислительных затратах.

Поставленная цель сравнительного исследования подходов к построению и обучению моделей NMT достигнута: рассмотрены основные архитектуры и методы обучения, подготовлена экспериментальная схема, выполнено сопоставление моделей и получена итоговая конфигурация, демонстрирующая устойчивое качество перевода на русско-английском направлении перевода.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Kocmi T., Avramidis E., Bawden R., Bojar O., Dvorkovich A., Federmann C., Fishel M., Freitag M., Gowda T., Grundkiewicz R. et al. Findings of the WMT24 General Machine Translation Shared Task: The LLM Era Is Here but MT Is Not Solved Yet [Электронный ресурс] // Proceedings of the Ninth Conference on Machine Translation. — 2024. — P. 1–46. — URL: <https://aclanthology.org/2024.wmt-1.1/>
2. Luong M.-T., Pham H., Manning C.D. Effective Approaches to Attention-based Neural Machine Translation [Электронный ресурс]. — 2015. — URL: <https://arxiv.org/abs/1508.04025>
3. Vaswani A., Shazeer N., Parmar N., Uszkoreit J., Jones L., Gomez A.N., Kaiser Ł., Polosukhin I. Attention Is All You Need [Электронный ресурс]. — 2017. — URL: <https://arxiv.org/abs/1706.03762>
4. Gu A., Dao T. Mamba: Linear-Time Sequence Modeling with Selective State Spaces [Электронный ресурс]. — 2023. — URL: <https://arxiv.org/abs/2312.00752>
5. Finkelstein M., Vilar D., Freitag M. Introducing the NewsPaLM MBR and QE Dataset: LLM-Generated High-Quality Parallel Data Outperforms Traditional Web-Crawled Data [Электронный ресурс] // Proceedings of the Ninth Conference on Machine Translation. — 2024. — URL: <https://aclanthology.org/2024.wmt-1.126/>
6. Sennrich R., Haddow B., Birch A. Neural Machine Translation of Rare Words with Subword Units [Электронный ресурс]. — 2016. — URL: <https://arxiv.org/abs/1508.07909>
7. Schuster M., Nakajima K. Japanese and Korean voice search [Электронный ресурс] // 2012 IEEE International Conference on Acoustics, Speech and Signal Processing. — 2012. — P. 5149–5152. — URL: <https://doi.org/10.1109/ICASSP.2012.6289079>

8. Kudo T. Subword Regularization: Improving Neural Network Translation Models with Multiple Subword Candidates [Электронный ресурс]. — 2018. — URL: <https://arxiv.org/abs/1804.10959>
9. Papineni K., Roukos S., Ward T., Zhu W.-J. BLEU: a Method for Automatic Evaluation of Machine Translation [Электронный ресурс] // Proceedings of the 40th Annual Meeting of the Association for Computational Linguistics. — 2002. — P. 311–318. — URL: <https://aclanthology.org/P02-1040/>
10. Popović M. chrF: character n-gram F-score for automatic MT evaluation [Электронный ресурс] // Proceedings of the Tenth Workshop on Statistical Machine Translation. — 2015. — P. 392–395. — URL: <https://aclanthology.org/W15-3049/>
11. Rei R., Stewart C., Farinha A.C., Lavie A. COMET: A Neural Framework for MT Evaluation [Электронный ресурс] // Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing. — 2020. — P. 2685–2702. — URL: <https://aclanthology.org/2020.emnlp-main.213/>
12. Holtzman A., Buys J., Du L., Forbes M., Choi Y. The Curious Case of Neural Text Degeneration [Электронный ресурс]. — 2019. — URL: <https://arxiv.org/abs/1904.09751>
13. Shao Z., Wang P., Zhu Q., Xu R., Song J., Bi X., Zhang H., Zhang M., Li Y.K., Wu Y. et al. DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models [Электронный ресурс]. — 2024. — URL: <https://arxiv.org/abs/2402.03300>
14. Wu L., Tian F., Qin T., Lai J., Liu T.-Y. A Study of Reinforcement Learning for Neural Machine Translation [Электронный ресурс] // Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing. — 2018. — P. 3612–3621. — URL: <https://aclanthology.org/D18-1397/>
15. Srivastava N., Hinton G., Krizhevsky A., Sutskever I., Salakhutdinov R. Dropout: A Simple Way to Prevent Neural Networks from Overfitting [Электронный ресурс] // Journal of Machine Learning Research. — 2014. — Vol. 15. — P. 1929–1958. — URL: <https://jmlr.org/papers/v15/srivastava14a.html>

16. Goodfellow I., Bengio Y., Courville A. Deep Learning [Электронный ресурс]. — 2016. — URL: <https://www.deeplearningbook.org/>
17. Loshchilov I., Hutter F. Decoupled Weight Decay Regularization [Электронный ресурс]. — 2017. — URL: <https://arxiv.org/abs/1711.05101>
18. Szegedy C., Vanhoucke V., Ioffe S., Shlens J., Wojna Z. Rethinking the Inception Architecture for Computer Vision [Электронный ресурс]. — 2015. — URL: <https://arxiv.org/abs/1512.00567>
19. Hochreiter S., Schmidhuber J. Long Short-Term Memory [Электронный ресурс] // Neural Computation. — 1997. — Vol. 9. No. 8. — P. 1735–1780. — URL: <https://doi.org/10.1162/neco.1997.9.8.1735>
20. Sutskever I., Vinyals O., Le Q.V. Sequence to Sequence Learning with Neural Networks [Электронный ресурс]. — 2014. — URL: <https://arxiv.org/abs/1409.3215>
21. Su J., Lu Y., Pan S., Murtadha A., Wen B., Liu Y. RoFormer: Enhanced Transformer with Rotary Position Embedding [Электронный ресурс]. — 2021. — URL: <https://arxiv.org/abs/2104.09864>
22. Zhang B., Williams P., Titov I., Sennrich R. Improving Massively Multilingual Neural Machine Translation and Zero-Shot Translation [Электронный ресурс] // Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics. — 2020. — P. 1628–1639. — URL: <https://aclanthology.org/2020.acl-main.148/>
23. Yandex LLC. License Agreement for the Use of the English-Russian Parallel Corpus [Электронный ресурс]. — URL: <https://yandex.ru/legal/corpus/en/?lang=en>
24. Esplà M., Forcada M., Ramírez-Sánchez G., Hoang H. ParaCrawl: Web-scale parallel corpora for the languages of the EU [Электронный ресурс] // Proceedings of Machine Translation Summit XVII: Translator, Project and User Tracks. — 2019. — P. 118–119. — URL: <https://aclanthology.org/W19-6721/>
25. Ziemski M., Junczys-Dowmunt M., Pouliquen B. The United Nations Parallel Corpus v1.0 [Электронный ресурс] // Proceedings of the Tenth International

- Conference on Language Resources and Evaluation. — 2016. — P. 3530–3534.
— URL: <https://aclanthology.org/L16-1561/>
26. Lui M., Baldwin T. langid.py: An Off-the-shelf Language Identification Tool [Электронный ресурс] // Proceedings of the ACL 2012 System Demonstrations. — 2012. — P. 25–30. — URL: <https://aclanthology.org/P12-3005/>
27. Rei R., Treviso M., Guerreiro N.M., Zerva C., Farinha A.C., Maroti C., de Souza J.G.C., Glushkova T., Alves D.M., Lavie A., Coheur L., Martins A.F.T. CometKiwi: IST-Unbabel 2022 Submission for the Quality Estimation Shared Task [Электронный ресурс]. — 2022. — URL: <https://arxiv.org/abs/2209.06243>
28. Paszke A., Gross S., Massa F., Lerer A., Bradbury J., Chanan G., Killeen T., Lin Z., Gimelshein N., Antiga L. et al. PyTorch: An Imperative Style, High-Performance Deep Learning Library [Электронный ресурс]. — 2019. — URL: <https://arxiv.org/abs/1912.01703>
29. Loshchilov I., Hutter F. SGDR: Stochastic Gradient Descent with Warm Restarts [Электронный ресурс]. — 2016. — URL: <https://arxiv.org/abs/1608.03983>
30. Kalamkar D., Mudigere D., Mellempudi N., Das D., Banerjee K., Avancha S., Vooturi D.T., Jammalamadaka N., Huang J., Yuen H. et al. A Study of BFLOAT16 for Deep Learning Training [Электронный ресурс]. — 2019. — URL: <https://arxiv.org/abs/1905.12322>
31. Smith S.L., Kindermans P.-J., Ying C., Le Q.V. Don't Decay the Learning Rate, Increase the Batch Size [Электронный ресурс]. — 2017. — URL: <https://arxiv.org/abs/1711.00489>
32. van Laarhoven T. L2 Regularization versus Batch and Weight Normalization [Электронный ресурс]. — 2017. — URL: <https://arxiv.org/abs/1706.05350>

ПРИЛОЖЕНИЕ А

Таблица А.1 – Базовая конфигурация Transformer encoder-decoder модели

Параметр	Значение
Архитектура	Transformer encoder-decoder
Тип токенизатора	bilingual unigram или bilingual BPE
Размер словаря	48 тыс. токенов
Позиционное представление	RoPE
Число слоев encoder	4
Число слоев decoder	4
Hidden size	256
Число attention-голов	8
FFN inner multiplier	2,67
Dropout	0,1
RoPE theta	10000

Таблица А.2 – Конфигурации моделей для сравнения PE и RoPE

Параметр	Модель с PE	Модель с RoPE
Архитектура	Transformer encoder-decoder	Transformer encoder-decoder
Позиционное представление	absolute positional encoding	rotary positional embeddings
Число слоев encoder	8	8
Число слоев decoder	8	8
Hidden size	768	768
Число attention-голов	12	12
FFN inner multiplier	3	3
Dropout	0,05	0,05
RoPE theta	—	10000
Токенизаторы	unigram	unigram

Таблица А.3 – Конфигурация GRPO-эксперимента

Параметр	Значение
Базовая архитектура	Transformer encoder-decoder
Токенизатор	bilingual unigram
Позиционное представление	RoPE
Число слоев encoder	8
Число слоев decoder	8
Hidden size	768
Число attention-голов	12
FFN inner multiplier	2
RoPE theta	10000
Размер группы GRPO	4 варианта перевода
Reward	$\frac{1}{3} \cdot \frac{\text{chrF}}{60} + \frac{2}{3} \cdot \frac{\text{COMET}}{0,845}$
Число обработанных примеров	около 50 тыс.

Таблица А.4 – Конфигурация модели для экспериментов с label smoothing

Параметр	Значение
Архитектура	Transformer encoder-decoder
Токенизатор исходного языка	unigram
Токенизатор целевого языка	unigram
Число слоев encoder	6
Число слоев decoder	6
Hidden size	768
Число attention-голов	16
FFN inner multiplier	2
Dropout	0,05
Максимальная длина входной последовательности	2048

Листинг А.1 – Реализация лучшей Transformer encoder-decoder модели

```

1  class TransformerEncoder(nn.Module):
2      def __init__(self, d_model: int, num_heads: int, dropout: float,
3                  inner_multiplier: float = 2.0, theta: float = 10000.0):
4          super().__init__()
5          self.attention = SelfAttention(
6              d_model, num_heads, dropout, theta=theta, causal=False
7          )
8          self.ffn = FFN(d_model, int(d_model * inner_multiplier))
9
10     def forward(self, x: Tensor, mask: Tensor):
11         x = x + self.attention(x, mask)
12         x = x + self.ffn(x)
13         return x
14
15
16     class TransformerDecoder(nn.Module):
17         def __init__(self, d_model: int, num_heads: int, dropout: float,
18                     inner_multiplier: float = 2.0, theta: float = 10000.0):
19             super().__init__()
20             self.self_attn = SelfAttention(
21                 d_model, num_heads=num_heads, dropout=dropout,
22                 theta=theta, causal=True,
23             )
24             self.cross_attn = CrossAttention(

```

```

25         d_model, num_heads=num_heads, dropout=dropout,
26     )
27     self.ffn = FFN(d_model, int(d_model * inner_multiplier))
28
29     def forward(self, x: Tensor, y: Tensor, mask: Tensor):
30         x = x + self.self_attn(x)
31         x = x + self.cross_attn(x, y, mask)
32         x = x + self.ffn(x)
33         return x
34
35
36 class CompleteTransformerSeq2Seq(EncoderDecoderBilingual):
37     def __init__(self, vocab_size: int, pad_token_id: int,
38                 bos_token_id: int, eos_token_id: int,
39                 encoder_layers: int, decoder_layers: int,
40                 d_model: int, num_heads: int, dropout: float,
41                 inner_multiplier: float, theta: float):
42         super().__init__(vocab_size, pad_token_id, bos_token_id, eos_token_id)
43         self.embedding = nn.Embedding(vocab_size, d_model, pad_token_id)
44         self.norm = RMSNorm(d_model)
45         self.head = nn.Linear(d_model, vocab_size)
46         self.encoder = nn.ModuleList([
47             TransformerEncoder(d_model, num_heads, dropout,
48                               inner_multiplier, theta)
49             for _ in range(encoder_layers)
50         ])
51         self.decoder = nn.ModuleList([
52             TransformerDecoder(d_model, num_heads, dropout,
53                               inner_multiplier, theta)
54             for _ in range(decoder_layers)
55         ])
56
57     def encode(self, input_ids: Tensor, key_padding_mask: Tensor) -> Tensor:
58         y = self.embedding(input_ids)
59         for layer in self.encoder:
60             y = layer(y, key_padding_mask)
61         return y
62
63     def decode(self, output_ids: Tensor, encoder_outputs: Tensor,
64               key_padding_mask: Tensor) -> Tensor:
65         x = self.embedding(output_ids)
66         for layer in self.decoder:
67             x = layer(x, encoder_outputs, key_padding_mask)
68         return self.head(self.norm(x))
69
70     def forward(self, input_ids: Tensor, output_ids: Tensor,
71                 attention_mask: Tensor) -> Tensor:
72         key_padding_mask = ~attention_mask.bool()

```

```

73         encoder_outputs = self.encode(input_ids, key_padding_mask)
74         return self.decode(output_ids, encoder_outputs, key_padding_mask)

```

Листинг А.2 – Реализация multi-head attention с FlashAttention

```

1  class SelfAttention(nn.Module):
2      def __init__(self, d_model: int, num_heads: int, dropout: float,
3                  causal: bool = False, theta: float = 10000.0):
4          super().__init__()
5          if d_model % num_heads != 0:
6              raise ValueError("d_model must be divisible by num_heads")
7
8          self.num_heads = num_heads
9          self.head_dim = d_model // num_heads
10         self.dropout = dropout
11         self.causal = causal
12         self.rope = RotaryEmbedding(self.head_dim, theta=theta)
13
14         self.q_proj = nn.Linear(d_model, d_model, bias=False)
15         self.k_proj = nn.Linear(d_model, d_model, bias=False)
16         self.v_proj = nn.Linear(d_model, d_model, bias=False)
17         self.o_proj = nn.Linear(d_model, d_model, bias=False)
18         self.norm = RMSNorm(d_model)
19
20     def forward(self, x: Tensor, mask: Tensor | None = None) -> Tensor:
21         batch_size, seq_length, d_model = x.shape
22         x = self.norm(x)
23
24         q = self.q_proj(x).view(batch_size, seq_length,
25                                 self.num_heads, self.head_dim)
26         k = self.k_proj(x).view(batch_size, seq_length,
27                                 self.num_heads, self.head_dim)
28         v = self.v_proj(x).view(batch_size, seq_length,
29                                 self.num_heads, self.head_dim)
30
31         cos, sin = self.rope(seq_length, x.device, q.dtype)
32         q = self.rope.rotate(q, cos, sin)
33         k = self.rope.rotate(k, cos, sin)
34
35         if mask is None:
36             x = flash_attn_func(
37                 q, k, v,
38                 dropout_p=self.dropout if self.training else 0.0,
39                 causal=self.causal,
40             )
41         else:
42             attention_mask = ~mask
43             q, indices, cu_seqlens, max_seqlen, _ = unpad_input(q, attention_mask
)

```

```

44         k, _, _, _ = unpad_input(k, attention_mask)
45         v, _, _, _ = unpad_input(v, attention_mask)
46         x = flash_attn_varlen_func(
47             q, k, v,
48             cu_seqlens_q=cu_seqlens,
49             cu_seqlens_k=cu_seqlens,
50             max_seqlen_q=max_seqlen,
51             max_seqlen_k=max_seqlen,
52             dropout_p=self.dropout if self.training else 0.0,
53             causal=self.causal,
54         )
55         x = pad_input(x, indices, batch_size, seq_length)
56
57     x = x.contiguous().view(batch_size, seq_length, d_model)
58     return self.o_proj(x)

```

Листинг А.3 – Реализация RoPE

```

1  class RotaryEmbedding(nn.Module):
2      inv_freq: Tensor
3
4      def __init__(self, dim: int, theta: float = 10000.0):
5          super().__init__()
6          if dim % 2 != 0:
7              raise ValueError("RoPE head dimension must be even")
8
9          inv_freq = 1.0 / (theta ** (torch.arange(0, dim, 2).float() / dim))
10         self.register_buffer("inv_freq", inv_freq, persistent=False)
11
12     def forward(self, seq_length: int, device: torch.device,
13                 dtype: torch.dtype) -> tuple[Tensor, Tensor]:
14         positions = torch.arange(seq_length, device=device,
15                                 dtype=self.inv_freq.dtype)
16         freqs = torch.outer(positions, self.inv_freq.to(device=device))
17         cos = freqs.cos().to(dtype=dtype)[None, :, None, :]
18         sin = freqs.sin().to(dtype=dtype)[None, :, None, :]
19         return cos, sin
20
21     def rotate(self, x: Tensor, cos: Tensor, sin: Tensor) -> Tensor:
22         x1 = x[..., 0::2]
23         x2 = x[..., 1::2]
24         return torch.stack(
25             (x1 * cos - x2 * sin, x1 * sin + x2 * cos),
26             dim=-1,
27         ).flatten(-2)

```

Листинг А.4 – Реализация absolute positional encoding

```

1  class PositionalEncoding(nn.Module):

```

```

2     pe: Tensor
3
4     def __init__(self, d_model: int, max_length: int = 5000):
5         super().__init__()
6         position = torch.arange(max_length).unsqueeze(1)
7         div_term = torch.exp(
8             torch.arange(0, d_model, 2) * (-math.log(10000.0) / d_model)
9         )
10
11        pe = torch.zeros(max_length, d_model)
12        pe[:, 0::2] = torch.sin(position * div_term)
13        pe[:, 1::2] = torch.cos(position * div_term[: pe[:, 1::2].shape[1]])
14        self.register_buffer("pe", pe.unsqueeze(0))
15
16    def forward(self, x: Tensor):
17        return x + self.pe[:, : x.size(1)].to(dtype=x.dtype)

```

Листинг А.5 – Разделение параметров для weight decay

```

1 def split_decay_params(model: nn.Module):
2     decay = []
3     no_decay = []
4
5     for name, param in model.named_parameters():
6         name = name.lower()
7         if name.endswith(".bias") or ("norm" in name) or ("embedding" in name):
8             no_decay.append(param)
9         else:
10            decay.append(param)
11
12    return decay, no_decay
13
14
15 decay, no_decay = split_decay_params(model)
16 optimizer = torch.optim.AdamW(
17     [
18         {"params": decay, "weight_decay": args.weight_decay},
19         {"params": no_decay, "weight_decay": 0.0},
20     ],
21     lr=args.lr,
22     betas=(args.beta1, args.beta2),
23 )

```

Листинг А.6 – Создание scheduler в PyTorch

```

1 warmup_scheduler = optim.lr_scheduler.LinearLR(
2     optimizer,
3     start_factor=0.01,
4     end_factor=1.0,

```

```

5     total_iters=args.warmup_steps,
6 )
7
8 cosine_scheduler = optim.lr_scheduler.CosineAnnealingLR(
9     optimizer,
10    T_max=(args.steps - args.warmup_steps),
11    eta_min=args.min_lr,
12 )
13
14 scheduler = optim.lr_scheduler.SequentialLR(
15     optimizer,
16     schedulers=[warmup_scheduler, cosine_scheduler],
17     milestones=[args.warmup_steps],
18 )

```

Листинг А.7 – Вычисление GRPO loss

```

1 def grpo_loss(
2     per_token_logps: Tensor,
3     old_per_token_logps: Tensor,
4     ref_per_token_logps: Tensor,
5     mask: Tensor,
6     advantages: Tensor,
7     clip_eps: float,
8     kl_beta: float,
9 ) -> tuple[Tensor, dict[str, Any]]:
10     advantages = advantages.detach().view(-1, 1).to(dtype=per_token_logps.dtype)
11     mask = mask.to(dtype=per_token_logps.dtype)
12
13     per_token_ratio = torch.exp(per_token_logps - old_per_token_logps)
14     per_token_clipped_ratio = per_token_ratio.clamp(1.0 - clip_eps, 1.0 + clip_eps)
15     per_token_policy_loss = -torch.min(
16         per_token_ratio * advantages,
17         per_token_clipped_ratio * advantages,
18     )
19
20     kl_delta = torch.clamp(ref_per_token_logps - per_token_logps,
21                             min=-20.0, max=20.0)
22     per_token_kl = torch.exp(kl_delta) - kl_delta - 1.0
23
24     sequence_token_count = mask.sum(dim=1).clamp_min(1.0)
25     policy_loss = ((per_token_policy_loss * mask).sum(dim=1)
26                    / sequence_token_count).mean()
27     kl = ((per_token_kl * mask).sum(dim=1) / sequence_token_count).mean()
28     loss = policy_loss + kl_beta * kl
29
30     return loss, {
31         "policy_loss": float(policy_loss.detach().cpu()),

```

```

32         "kl": float(kl.detach().cpu()),
33     }

```

Листинг А.8 – Генерация с temperature и top-p sampling

```

1  def apply_inference_params(
2      logits: Tensor,
3      temperature: float = 0.0,
4      top_p: float = 1.0,
5  ) -> Tensor:
6      logits = logits.float()
7      if temperature > 0.0:
8          logits = logits / temperature
9
10     if top_p == 1.0:
11         return logits
12
13     sorted_logits, sorted_indices = torch.sort(logits, descending=False, dim=-1)
14     cumulative_probabilities = sorted_logits.softmax(dim=-1).cumsum(dim=-1)
15     sorted_indices_to_remove = cumulative_probabilities <= (1 - top_p)
16     sorted_indices_to_remove[..., -1:] = False
17
18     indices_to_remove = torch.zeros_like(sorted_indices_to_remove).scatter(
19         -1,
20         sorted_indices,
21         sorted_indices_to_remove,
22     )
23     return logits.masked_fill(indices_to_remove, float("-inf"))
24
25
26  def sample_next_token(
27      logits: Tensor,
28      temperature: float = 0.0,
29      top_p: float = 1.0,
30  ) -> Tensor:
31      if temperature == 0.0:
32          return logits.argmax(dim=-1)
33
34      logits = apply_inference_params(logits, temperature, top_p)
35      probabilities = torch.softmax(logits, dim=-1)
36      sampled = torch.multinomial(
37          probabilities.reshape(-1, probabilities.size(-1)),
38          num_samples=1,
39      )
40      return sampled.reshape(probabilities.shape[:-1])

```